

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Software Engineering 2 Requirements Analysis and Specification Document

Author(s): **Francesco Rausa - 10833982**
David Ravelli - 10843295
Pietro Poggi - 10891051

Academic Year: 2025-2026
Version: 2.0
Release date: 07/01/2026

Contents

Contents **i**

1 Introduction **1**
1.1 Purpose . . . 1
1.1.1 Goals . . . 1
1.2 Scope . . . 2
1.2.1 World Phenomena . . . 2
1.2.2 Shared Phenomena . . . 3
1.3 Definitions, Acronyms, Abbreviations . . . 3
1.3.1 Definitions . . . 4
1.3.2 Acronyms . . . 4
1.3.3 Abbreviations . . . 4
1.4 Revision history . . . 5
1.5 Reference Documents . . . 5
1.6 Document Structure . . . 5

2 Overall Description **6**
2.1 Product Perspective . . . 6
2.1.1 Scenarios . . . 6
2.1.2 Domain-level Class Diagram . . . 8
2.1.3 State Diagrams . . . 11
2.2 Product Functions . . . 14
2.3 Requirements . . . 16
2.4 User Characteristics . . . 19
2.5 Assumptions, Dependencies and Constraints . . . 20
2.5.1 Domain Assumptions . . . 20

3 Specific Requirements **22**
3.1 External Interface Requirements . . . 22
3.1.1 User Interfaces . . . 22
3.1.2 Hardware Interfaces . . . 29
3.1.3 Software Interfaces . . . 29
3.1.4 Communication Interfaces . . . 30
3.2 Functional Requirements . . . 31
3.2.1 Use Cases and Activity Diagrams . . . 32

3.2.2 Mapping On Requirements . . . 61
3.3 Performance Requirements . . . 62
3.4 Design Constraints . . . 63
3.5 Software System Attributes . . . 64

4 Formal Analysis using Alloy 66
4.1 Objectives . . . 66
4.1.1 Consistent System Evolution . . . 66
4.1.2 Contribution and Detection Management Logic . . . 66
4.2 The Model . . . 67
4.2.1 Modular Structure . . . 67
4.2.2 Signatures . . . 68
4.2.3 Predicates . . . 71
4.2.4 Facts . . . 74
4.2.5 Main Module . . . 78
4.3 Validation through the Model . . . 84
4.3.1 Scenario: User Reporting a Road Issue . . . 84
4.3.2 Scenario: Automatic Detection Confirmation . . . 85
4.3.3 Scenario: Path Invalidation and Independence . . . 86

5 Effort Spent 88

6 References 89
6.1 Used Tools . . . 89

List of Figures 90

1 Introduction

1.1. Purpose

In recent years, the use of bicycles has significantly increased, both as a means of daily transportation and as part of urban sustainability initiatives. However, despite this growing interest, it remains difficult for cyclists to access reliable and up-to-date information about existing bike paths, their conditions, and potential obstacles along the way. Such a lack of information often makes it challenging to plan safe, efficient, and enjoyable routes, discouraging many users from adopting cycling as a regular mode of transport.

Best Bike Paths (BBP) is designed to overcome these limitations by providing a collaborative and intelligent platform that collects, analyzes, and shares data about cycling routes. Through the combination of user-generated content and automated data collection from mobile sensors, BBP enables users to record their trips, monitor their performance, and contribute to a shared, continuously updated map of bike paths.

The system also integrates external weather information to enhance the understanding of riding conditions, and it supports recommendation features to suggest the best paths between two points based on quality, safety, and user preferences.

1.1.1. Goals

In this section, there are defined the goals that the system has to achieve:

Profile Management and Personal Tracking

[G1] Registered users should be able to manage their profile on the platform.

[G2] Registered users should be able to record and store their bike trips.

Bike Path Information Insertion and Management

[G3] The system should provide detailed statistics about recorded trips (distance, average speed, performance metrics).

[G4] Registered users should be able to manually insert information about bike paths.

[G5] Registered users should be able to automatically insert information about bike paths through data acquisition from mobile devices.

[G6] Users should be able to publish paths' information.

Consultation

[G7] Both registered and unregistered users should be able to consult information about available bike paths and choose the most suitable one according to their preferences.

Data Aggregation

[G8] The system should aggregate information coming from multiple users about the same paths, considering data freshness and number of confirmations representing them as accurately as possible based on users' contributions, allowing them to discover more comfortable routes than generic roads .

1.2. Scope

Best Bike Paths (BBP) is a platform that supports cyclists in creating and consulting a collaborative inventory of bike paths. The platform manages the path status through standardized categories (optimal, medium, sufficient, requires maintenance) and tracks the presence of relevant obstacles such as potholes or other issues that may compromise safety.

BBP provides the following core functionalities:

- Support cyclists in recording and storing their trips with performance statistics.
- Enable manual and automatic insertion of bike path information collected from mobile devices.
- Allow users (registered and unregistered) to explore bike paths on a map between chosen origin and destination points.
- Aggregate and reconcile path data from multiple users, considering temporal proximity and consensus.
- Enrich trip data with meteorological information obtained from external services.

1.2.1. World Phenomena

[WP1] Cyclists travel on roads and bike paths by bicycle.

[WP2] The conditions of roads and bike paths change over time.

[WP3] Cyclists encounter physical obstacles or hazards along bike paths.

[WP4] Different cyclists use the same bike path at different times, each experiencing its conditions independently.

[WP5] Cyclists observe and formulate opinions on the status of paths they travel.

[WP6] External meteorological services collect data on weather conditions, temperature, and wind speed.

1.2.2. Shared Phenomena

World Controlled

[SP1] Users sign up to the platform providing their credentials.

[SP2] Users log into the platform.

[SP3] Registered users manually start and stop the recording of a trip.

[SP4] Users grant permission for BBP to access sensor data during trips.

[SP5] Registered users manually insert information about bike paths.

[SP6] Registered users confirm or correct information automatically acquired by BBP.

[SP7] Registered authorize the publication of their submitted path information.

[SP8] Users specify an origin and a destination to search for bike paths.

[SP9] Users request detailed path information.

[SP10] Users (including non-registered ones) navigate the map of available paths.

[SP11] Cyclists' mobile devices detect movement, GPS position, and accelerations during trips.

Machine Controlled

[SP12] The system reconstructs the path followed by the user from acquired GPS data and shows it to the user.

[SP13] The system presents automatically acquired information to the user for confirmation/-correction.

[SP14] The system requests meteorological information from the external service for a specific trip.

[SP15] The system shows identified paths on a map ordered by score.

[SP16] The system shows the user the statistics of their stored trips.

[SP17] The system displays the requested information.

1.3. Definitions, Acronyms, Abbreviations

1.3.1. Definitions

BBP (Best Bike Paths) The collaborative platform, subject of this document, designed to collect, analyze, and share data about cycling routes.

Registered User A user who has created a profile on the platform and can record trips, insert data, and manage their profile.

Unregistered User A user who can only consult information and view maps of available bike paths.

Path Status A standardized rating of a path's condition (e.g., optimal, medium, sufficient, requires maintenance).

Trip The recording of a single bicycle journey, which includes GPS data, sensor readings, and statistics.

StreetSegment The fundamental unit of the system, representing a stretch of road between two intersections or junctions. Each StreetSegment contains no intersections within it and constitutes the most atomic level of the road network.

Path / BikePath A sequence of connected StreetSegments selected to ensure safe and comfortable conditions for cyclists. A Path is considered suitable when the motor vehicle speed limit is reasonable for cycling use or when a dedicated bike lane is available.

1.3.2. Acronyms

BBP Best Bike Paths

G Goal

WP World Phenomena

SP Shared Phenomena

GPS Global Positioning System

API Application Programming Interface

DB Data Base

1.3.3. Abbreviations

Id Identifier

1.4. Revision history

Revised on	Version	Description
23/12/2025	1.0	Initial Release of the document
07/01/2026	2.0	Update of the document according to Design Document decisions

Table 1.1: Revision history

Integrations in Version 2.0

- Updated UC11 by specifying that the inference process is performed as a single step after the completion of the trip.
- Updated UI images color scheme and split the manual insertion functionalities.

1.5. Reference Documents

- [1] Di Nitto, Rossi, Camilli, *"A.Y. 2025-2026 Software Engineering 2 Requirement Engineering and Design Project"*, 2025.
- [2] Matteo Camilli, Elisabetta Di Nitto, Alfonso Fuggetta, Alessandro Margara, Matteo G. Rossi, Damian A. Tamburri, *Software Engineering: a Structured Journey from Inception to Delivery*, Bocconi University Press, 2025, ISBN: 9791280623508

1.6. Document Structure

This document is composed of six sections:

- 1st Chapter - Introduction
- 2nd Chapter - Overall Description
- 3rd Chapter - Specific Requirements
- 4th Chapter - Formal Analysis using Alloy
- 5th Chapter - Effort Spent
- 6th Chapter - References

2 Overall Description

2.1. Product Perspective

2.1.1. Scenarios

1st Scenario: User Registration and Profile Management
Elena Rossi, an avid cyclist and member of a local biking association, decides to join the BBP platform to keep track of her cycling activity and share information with other bikers. Upon opening the application, she selects the *Sign Up* option. BBP prompts her to enter her personal details such as name, email, and password. Once the verification process is complete, Elena accesses her personal profile page. She updates her information, adds a short description of her cycling interests, and enables location permissions. From this moment, Elena becomes a registered user, able to record her trips and contribute data about bike paths.

2nd Scenario: Recording and Viewing a Trip with Statistics
Fausto Coppi is training for his next Tour. Fausto decides to record his rides using BBP. Before starting, he opens the app and presses *Start Trip Recording*. During the ride, the app collects GPS data to track his route and speed. Once he arrives at his destination, he presses *Stop*. The system automatically saves his trip and calculates key statistics: total distance, average speed, and travel time. Fausto receives an automatic summary of the detected path and potential obstacles identified by the sensors. He notices that BBP has mistakenly flagged a bump as a pothole, so he corrects the information before confirming the path. Once validated, the path is saved in his account. Fausto later accesses his trip history, where he can review all his recorded rides, compare performances, and track his progress over time.

3rd Scenario: Manual Insertion of Path Information
During one of her daily rides, Federica Conti—an environmental engineering student and urban cyclist—remembers that a particular area of the city has no documented cycling paths on BBP's map. Curious and motivated to improve the platform's coverage, she decides to explore the zone herself. While riding through the neighborhood, Federica discovers a newly built cycling lane that is not yet available on BBP. Wanting to record it before she forgets, she opens the *Manual insertion section* and manually inserts all the relevant details: the street names, total distance, surface quality, and current condition (e.g., Optimal). She also reports a minor obstacle: a small pothole near a crossroad. However, due to the lack of internet connection in that area, she is unable to upload the contribution immediately. Instead, BBP's local storage feature allows her to save the path as a local draft associated with her account. Once she returns home and reconnects to the internet, the application automatically synchronizes the previously saved trip with her BBP account. After reviewing the details one more time, Federica decides to

make the path public. The system then updates the platform, publishing the new cycling lane for all users to see.

4th Scenario: Publishing a Recently Completed Path
After reviewing and confirming the data from her latest ride, Sofia Moretti, an experienced city biker, decides to share it immediately with other cyclists. As soon as she finishes the trip and validates the automatically collected information, BBP prompts her with an option to *Publish Path Information*. Sofia confirms the publication request, and the system instantly adds the generated contributions to the public database, updating , after the merge process, the collaborative map of available routes. Within moments, the new StreetSegments becomes visible to all users of the platform.

5th Scenario: Publishing a Previously Stored Path
Two weeks later, Andrea Ricci, a cycling enthusiast who often explores rural routes, revisits one of his older recorded trips, which he never published. He clicks *Publish*, and BBP processes the data, making it available in the shared inventory. The system automatically merges his contribution with existing information about the same route, if available, ensuring that the most recent and confirmed data are displayed.

6th Scenario: Consulting and Choosing a Path (Registered User)
Francesco Leone, a registered cyclist who commutes daily by bike, plans a weekend trip across the city. He logs into BBP and specifies his origin and destination. The system searches the database for suitable routes and displays multiple options on the map sorted by their score, considering factors such as safety, condition, and efficiency. Since Francesco's profile indicates a preference for smoother surfaces and minimal traffic, BBP's recommendation system automatically prioritizes such routes in the results. After reviewing the suggested paths, Francesco finds a possibly suitable paths and decides to follow it this weekend.

7th Scenario: Consulting and Choosing a Path (Unregistered User)
Marta Villa, a casual cyclist who enjoys leisure rides on weekends, wants to discover new paths without creating an account. She opens the BBP app without logging in and accesses the public map. BBP displays all available paths but without personalized recommendations. Marta browses the map, filters by *Optimal* status, and selects a route along the lakeside. Although she cannot record trips or add data, she can still view general information, such as total length, average condition, and user comments.

8th Scenario: Updating Information on an Existing Path
During a weekly commute, Tommaso Greco, a frequent user of BBP, rides along a route previously marked as *Optimal*. He notices that a section of the track has recently deteriorated and requires maintenance. Once he finishes the ride, he opens BBP and selects *Update Path*

Information. He searches for the existing StreetSegment and updates its condition from *Optimal* to *Requires Maintenance*. The system records his update and automatically aggregates it with reports from other users who have recently traveled the same route. By considering both the number of confirmations and the recency of the available data, BBP determines the most reliable current status of the path and updates its classification accordingly.

2.1.2. Domain-level Class Diagram

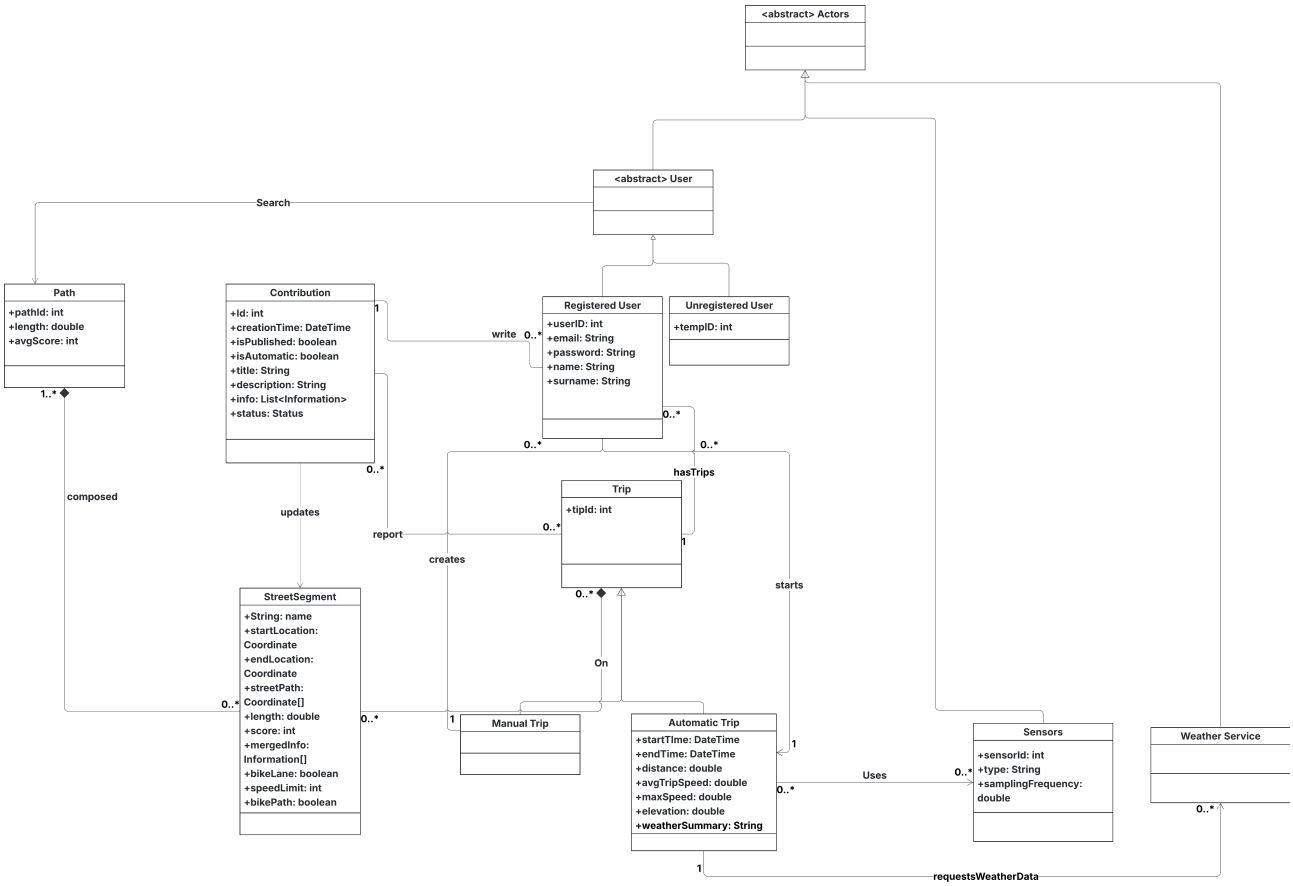


Figure 2.1: Domain-Level Class Diagram.

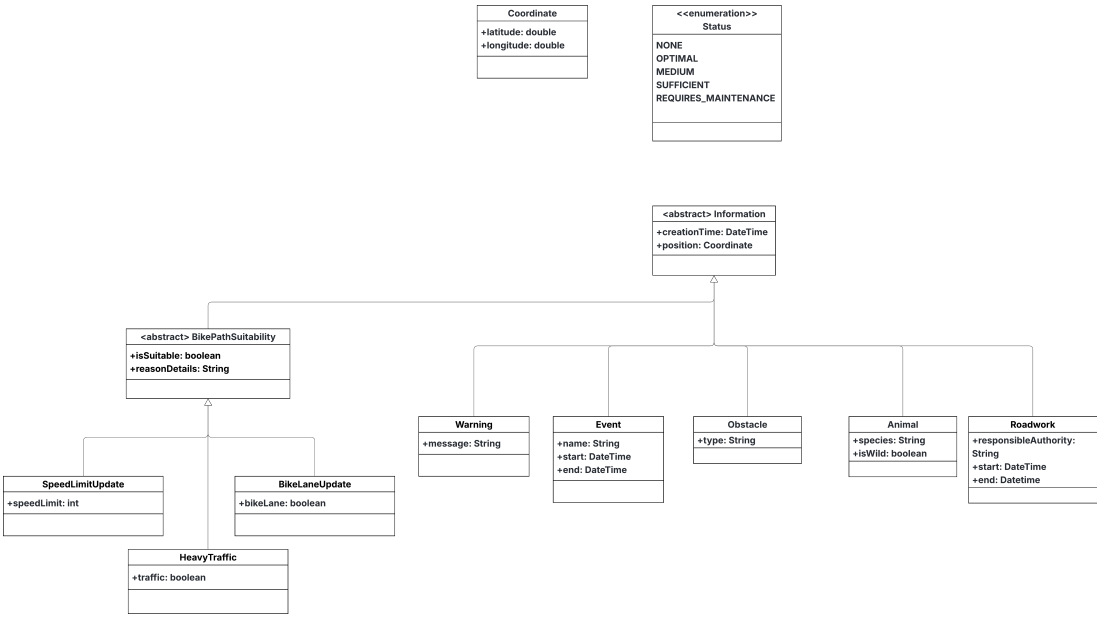


Figure 2.2: Domain-Level Class Diagram.

The entities interacting with the system are organized around a hierarchical user structure that reflects the different capabilities exposed by the application. At the top, the abstract *User* class generalizes every individual accessing the platform, while *RegisteredUser* extends it with attributes such as `userID`, `email`, `password`, `name`, and `surname`, enabling trip recording, contribution management, and profile customization. *UnregisteredUser*, still a specialization of *User*, represents visitors who can browse the map and search paths without owning trips or publishing contributions; both types of users interact with the map-based exploration of the cycling network, but only registered users are associated with a history of trips through the *Trip* entity.

The core dynamic element of the model is the *Trip* class, which represents a cycling session performed by a registered user and identified by `tripId`. Trips are specialized into *ManualTrip* and *AutomaticTrip*, depending on whether they are manually specified or recorded through sensors, with *AutomaticTrip* storing temporal and quantitative data such as `startTime`, `endTime`, `distance`, `avgTripSpeed`, `maxSpeed`, `elevation`, and an optional `weatherSummary`, and being associated with *Sensors* that provide GPS and motion readings. Each trip is mapped onto the infrastructure via one or more *StreetSegment* instances, modeling individual pieces of road between intersections, and for each traversed street segment the trip can generate at most one associated *Contribution*, resulting in a trip-level collection of contributions, each focusing on a single street segment crossed during the ride.

The *Path* class models higher-level routes composed of multiple *StreetSegment* entities, and is

characterized by attributes such as `pathId`, `length`, and `avgScore`. A street segment is defined through `name`, its endpoints `startLocation` and `endLocation` of type *Coordinate*, a geometric polyline `streetPath`, a local `score`, and a set of `mergedInfo` items that summarize aggregated information, including whether the segment is currently considered part of the cycling network via attributes such as `bikeLane`, `speedLimit`, and `bikePath`. In this model, a street segment is the atomic infrastructure unit: each *Contribution* always refers to exactly one *StreetSegment*, and the system periodically merges all contributions linked to that segment in order to update its score, its merged information, and ultimately its visibility in paths and map-based queries. When a segment becomes a bike path according to the merged data, it is highlighted on the map and becomes eligible to be part of the paths suggested to users when searching for routes between two locations.

A *Contribution* encapsulates any piece of information about the cycling network generated during trips or via manual reports. It includes metadata such as `id`, `creationTime`, `title`, `description`, and flags `isPublished` and `isAutomatic`, as well as an optional qualitative `status` of type *Status*, ranging from `NONE` to `OPTIMAL`, `MEDIUM`, `SUFFICIENT`, and `REQUIRES_MAINTENANCE`. Each contribution also maintains a list of *Information* elements (`info: List<Information>`) describing the actual content of the report; among these, a distinguished group is represented by the abstract *BikePathSuitability* subclass, which includes attributes `isSuitable` and `reasonDetails` and is further specialized into concrete classes such as *SpeedLimitUpdate*, *BikeLaneUpdate*, and *HeavyTraffic*. When a contribution concerns a street segment that is not currently marked as bike path, ordinary information items (for example obstacles or roadworks) are immediately discarded during merging, whereas *BikePathSuitability* information is temporarily retained: the system periodically retrieves all contributions from different users for that segment and only after a given time window, if multiple coherent suitability items support the same conclusion (for instance, consistent speed-limit or bike-lane updates), it may promote the segment to bike path instead of discarding them. In this way, bike-path suitability information acts as a special trigger influencing both the boolean `bikePath` attribute of the street segment and the computation of path suggestions, while generic information only refines the segment's score and maintenance status.

Infrastructure-related facts are modeled by the abstract *Information* class, which records `creationTime` and `position` (a *Coordinate* described by `latitude` and `longitude`), and is specialized not only by *BikePathSuitability* but also by other concrete subclasses. These include *Warning*, *Event*, *Obstacle*, *Animal*, and *Roadwork*, which respectively store a textual `message`, an event `name` and validity interval, an obstacle `type`, an `animal species` with a flag indicating whether it is wild, and the `responsibleAuthority` plus temporal bounds for roadworks. Such information items are attached to contributions and, after merging, to street segments as part of

`mergedInfo`, providing a structured and typed view of the network's conditions that supports both the computation of scores and the decision about whether a non-bike street segment can be upgraded to bike path.

Finally, the sensing and external-service layer is abstracted by the *Sensors* and *WeatherService* entities. *Sensors*, characterized by `sensorId`, `type`, and `samplingFrequency`, are used by automatic trips to acquire motion and positioning data, which underpin automatic contribution creation, obstacle inference, and interruption detection. The *WeatherService* represents an external provider contacted by the system to enrich automatic trips with contextual weather information, explaining the presence of the `weatherSummary` attribute in *AutomaticTrip*; together with user-generated contributions and special *BikePathSuitability* items, these elements allow the system to maintain an up-to-date, bike-centric representation of the road network that dynamically highlights and exploits suitable segments when answering path search requests.

2.1.3. State Diagrams

User Registration and Account Management

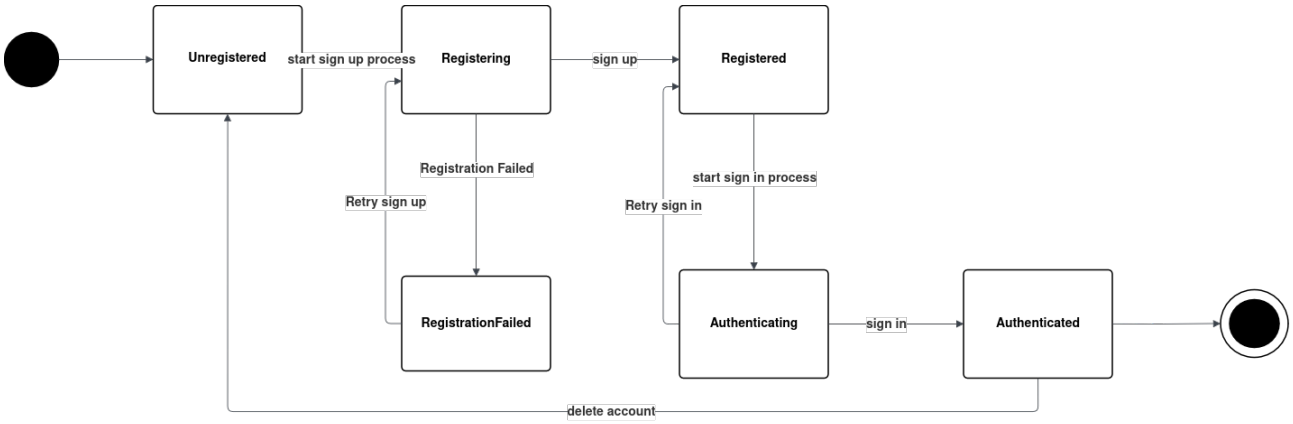


Figure 2.3: User Registration and Account Management.

This state diagram models the lifecycle of a user account within the application, from the initial unregistered state to registration, authentication, and eventual termination. The process begins with the user in the 'Unregistered' state. Upon initiating sign-up, the account transitions to 'Registering'; if details are submitted correctly, the user reaches 'Registered'. Failed registration attempts are captured in 'RegistrationFailed' with a loop for retrying or returning to the unregistered state. Similarly, the sign-in phase transitions through 'Authenticating', allowing for authentication retries or failures before advancing to the 'Authenticated' state when credentials are correct. At any point after registration, the user can choose to delete their account,

moving directly to the terminal state. This lifecycle ensures secure onboarding and persistent access control, while supporting error handling and explicit account dismissal.

Automatic Trip

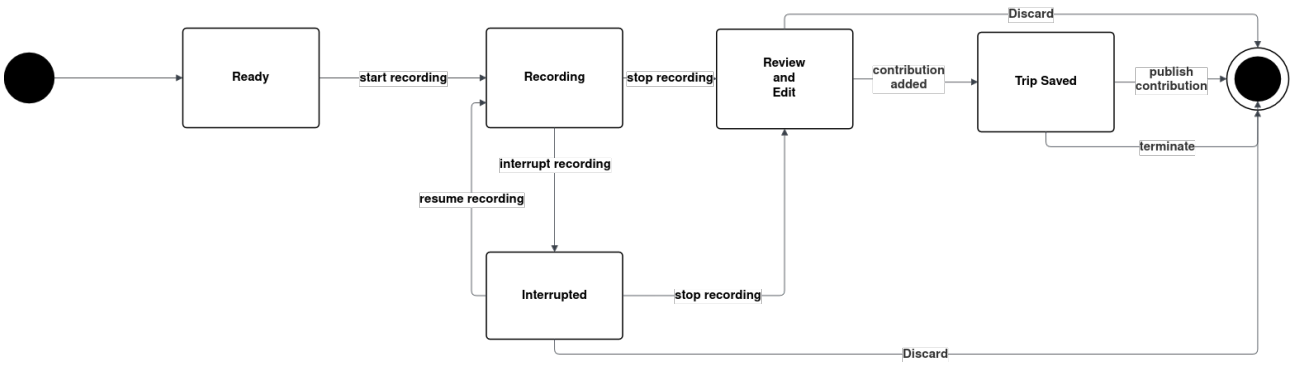


Figure 2.4: Automatic Trip.

The automatic trip workflow begins when the user simply presses a button to start recording. While the user is moving, the application continuously collects sensor and GPS data; if the user stops for a significant amount of time, the system automatically detects the lack of movement and pauses the recording, resuming it as soon as the ride restarts. At any moment, the user can explicitly terminate the trip by pressing the stop button, or decide to discard the recording, in which case no trip or contribution is stored. When a trip is successfully concluded, the system automatically generates a draft contribution populated with the information inferred from the recorded data. The user can review and edit this contribution, then either save it only in the personal trip history or publish it so that it becomes visible to other users; publication can also be postponed, allowing the user to keep the contribution private and decide later whether to make it public.

Manual Trip

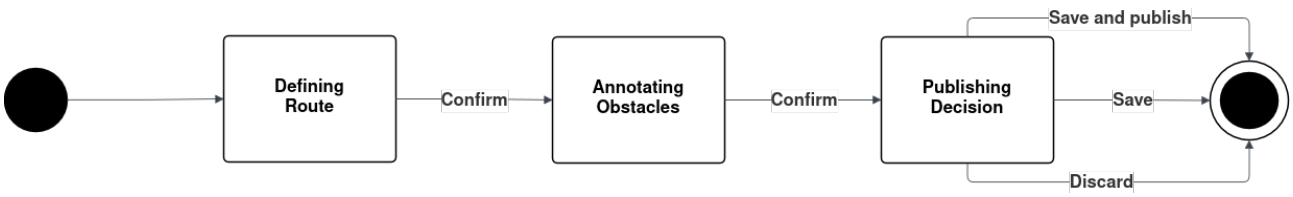


Figure 2.5: Manual Trip.

The manual trip workflow enables the user to specify not only the start and end points of their journey, but also the precise roads and paths actually taken during the ride. After selecting an origin and destination, the user can review the automatically proposed route and, if necessary, manually adjust it to match the streets or segments physically traversed. Throughout this process, the user is able to document issues encountered—such as potholes or other obstacles—along the exact path. For each manual trip, one or more manual contributions are generated, with the user deciding for each whether to publish it immediately or keep it private in their personal trip history, retaining the option to publish later. Unlike automatic trips, manual trips do not include sensor-based statistics (such as speed) or weather conditions, and focus solely on user-reported obstacles and qualitative route details.

StreetSegment

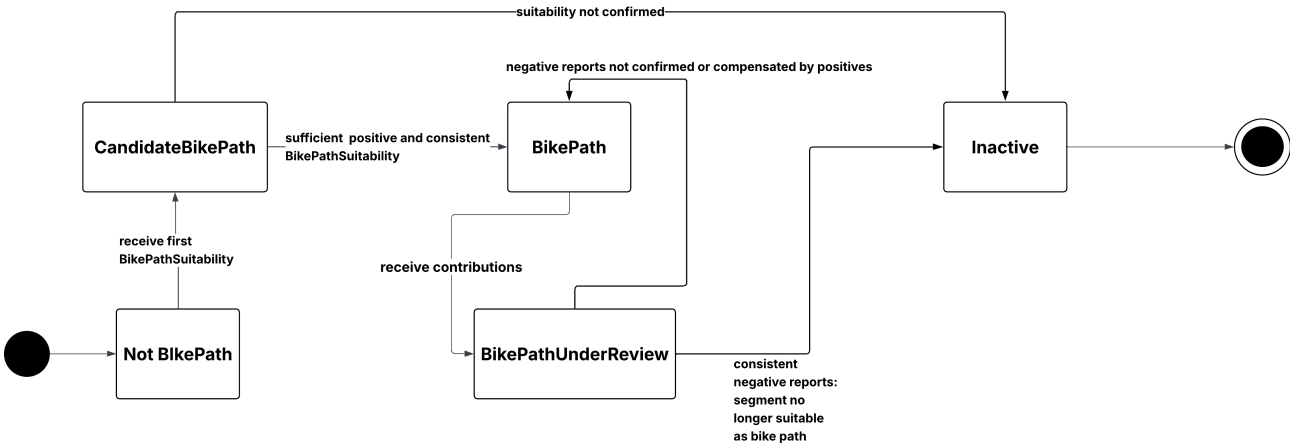


Figure 2.6: StreetSegment.

The street segment state diagram models how a road segment is progressively considered, monitored, and possibly dismissed as part of the cycling network. Each segment starts in the 'NotBikePath' state and moves to 'CandidateBikePath' when it receives the first BikePathSuitability information derived from user contributions. During an observation period, the system keeps collecting suitability information: if, at the end, the accumulated BikePathSuitability is sufficiently positive and consistent, the segment is promoted to the 'BikePath' state; otherwise, if suitability is not confirmed, the segment transitions directly to 'Inactive', meaning it is no longer considered for cycling purposes.

Once in the 'BikePath' state, the segment continues to receive contributions, including BikePathSuitability items and other Information such as obstacles or roadworks, and these trigger a transition to 'BikePathUnderReview' whenever an additional evaluation is required. In this review state, the system aggregates incoming suitability reports: if negative reports are not

confirmed or are compensated by positive ones, the segment returns to 'BikePath', updating its merged information with any new obstacles or events while still being considered suitable; if, instead, consistent negative reports indicate that the segment is no longer suitable as a bike path, it moves to the 'Inactive' state. A direct transition from 'BikePath' to 'Inactive' also exists to model administrative decisions or permanent closures, and even after a segment has become 'Inactive', the arrival of new BikePathSuitability information can restart the cycle by allowing it to be reconsidered as a 'CandidateBikePath'.

2.2. Product Functions

The BBP platform is designed to support cyclists in the collection, exploration, and sharing of information about bike paths. It provides functionalities that cover every stage of the cycling experience, from trip recording and path contribution to map consultation and route recommendation. By integrating automated sensor detection, user-generated data, and aggregation mechanisms, the platform ensures reliable, up-to-date information for all users, offering a seamless and collaborative riding experience.

- **Sign Up and Profile Management**
  Users create an account on the platform to access advanced functionalities, such as trip recording and contribution publishing. During registration, they provide personal information and configure basic preferences (e.g., cycling style or comfort priorities). Registered users can later update their profile, manage privacy settings, and modify their preferences, which are used by the recommendation system to tailor route suggestions.

- **Trip Recording and Storage**
  Registered users can record their bike trips through the automatic tracking option. During a ride, the system collects essential trip data such as GPS position, speed, and timestamps. Once the trip ends, the user can save it to their personal history, where BBP generates statistical summaries including total distance, average speed, travel time, and performance indicators. These statistics allow cyclists to review past activity and monitor their progress over time.

- **Manual Insertion of Path Information**
  Users can manually report details about bike paths that are missing or incomplete in the shared database. This includes route description, path condition (e.g., Optimal, Medium, Requires Maintenance), and obstacles encountered. The system stores submitted information as a contribution that can later be published to the public map.

- **Automatic Path Detection and Confirmation**

BBP supports automatic acquisition of path information through mobile sensors. When activated, the system uses GPS, accelerometer, and gyroscope data to reconstruct the path followed and detect potential obstacles such as potholes or irregularities. Detected information is presented to the user for review: registered users must confirm or correct each detection before it becomes a valid contribution.

- **Publication of Path Information**
  Registered users can publish both manually inserted data and confirmed automatic contributions. Publishing makes the information visible to all platform users and allows it to affect the global classification, condition score, and visualization of the corresponding bike path. Users may publish contributions immediately after a trip or at a later moment by reviewing stored entries.

- **Bike Path Consultation and Map Navigation**
  BBP enables both registered and unregistered users to browse a map of available cycling paths. Users can search for routes between a chosen origin and destination. The system displays viable paths, enriched with condition evaluations, obstacles, and contributions from the community. Maps remain accessible even without an account, though personalized features are limited to registered users.

- **Personalized Route Recommendation**
  The platform automatically generates a list of suitable paths between two points by analyzing path conditions, traffic comfort, safety indicators, and user preferences (for registered users only). The recommendation engine ranks each path by score, which considers contributions, surface smoothness, obstacle frequency, and how effectively the path leads to the destination. Registered users receive more personalized suggestions based on their saved profile settings.

- **Data Aggregation and Reconciliation**
  BBP aggregates contributions from multiple users who traveled the same path. The system applies a prevalence strategy, combining the number of confirmations, recency of reports, and consistency of information to derive the most reliable status for each path segment. This aggregated evaluation determines the official condition displayed on the map.

- **Weather Data Enrichment**
  For each recorded trip, BBP can request meteorological data from external services, such as weather conditions, temperature, and wind intensity. This information is integrated into the trip summary, enabling cyclists to better understand the context of their performance or encountered difficulties.

2.3. Requirements

User Authentication and Profile Management

[R1] Upon request, the system shall allow a new User to sign up to the platform, as long as all required personal information is provided and no existing account is associated with the submitted email.

[R2]Upon request, the system shall allow the User to log into the platform, as long as authentication succeeds, and access to his personal profile.

[R3]Upon request, the system shall allow a Registered User to update their profile information, as long as all necessary data is provided.

[R4]Whenever a Registered User updates their preferences, the system shall store these changes and apply them to future personalized route recommendations.

Trip Recording and Statistics

[R5]Upon request, the system shall allow a Registered User to manually start recording a trip, as long as GPS permissions are granted.

[R6]Upon request, the system shall allow a Registered User to manually stop a trip that was previously started.

[R7]Whenever an automatic trip is recorded, the system shall store the trip with its GPS track, timestamps, speed data and contributions into the User's trip history.

[R8]Whenever an Automatic Trip is stored, the system shall compute statistics such as total distance, average speed, travel time, and other performance metrics.

[R9] Whenever automatic detection identifies a potential obstacle, the system shall generate a detection item and store it in the User's trip session.

[R10] Upon completion of an automatic trip, the system shall present the detected path and all detected obstacles to the User, requesting confirmation or correction.

[R11] Whenever a Registered User confirms or corrects automatically detected information, the system shall convert such detections into contributions associated with the current path.

[R12] Upon request, the system shall allow a Registered User to manually create a Manual trip by specifying the StreetSegments they have cycled through, without using sensor data and without computing trip statistics.

[R13] Upon request, the system shall allow a Registered User to manually create a contribution describing information about a specific StreetSegment (e.g., obstacles or general condition), regardless of whether it is associated with a manually created trip.

[R14] Upon request, the system shall allow a Registered User to publish any of their manual contributions from a stored trip not yet published.

[R15]Upon request, the system shall show a Registered User the statistics and contributions of any of their stored trips.

Publication and Aggregation of Contributions

[R16] Upon request, the system shall allow a Registered User to publish any of their contributions (manual or confirmed automatic) that are currently in draft state.

[R17] Whenever a Registered User publishes a contribution, the system shall add all its Information items to the public dataset associated with the corresponding StreetSegment.

[R18] Whenever a recently completed automatic trip ends and its detections are validated, the system shall offer the User an immediate option to publish the contributions generated from that trip.

[R19] Whenever a published contribution contains standard Information (e.g., obstacles, roadworks, warnings) about a StreetSegment that is currently considered a bike path, the system shall merge such Information into the StreetSegment's aggregated data used for routing and scoring.

[R20] Whenever a published contribution contains BikePathSuitability Information about a StreetSegment, the system shall store it and consider it in the periodic aggregation of bike-path suitability for that StreetSegment, regardless of its current bike-path status.

[R21] At configurable time intervals, the system shall aggregate all published contributions referring to the same StreetSegment, taking into account data recency, number of supporting contributions, and type of Information, in order to update the StreetSegment's merged information, score, and bike-path status.

[R22] Whenever a published contribution contains only standard Information (e.g., obstacles, roadworks, warnings) about a StreetSegment that is not currently marked as a bike path, the system shall discard such Information for aggregation purposes and shall not use it to update that StreetSegment.

[R23] Whenever the aggregated BikePathSuitability Information for a StreetSegment exceeds a configurable positive threshold, the system shall mark that StreetSegment as a bike path;

whenever it falls below a configurable negative threshold, the system shall unmark that StreetSegment as a bike path.

Path Consultation and Map Navigation

[R24] Upon request, the system shall allow any User (registered or unregistered) to browse the map and visually inspect all StreetSegments currently marked as bike paths.

[R25] Upon request, the system shall allow the User to specify an origin and a destination for route lookup.

[R26] Whenever the system receives an origin and a destination, it shall compute candidate Paths by running a routing algorithm over the StreetSegments that are currently marked as bike paths.

[R27] Whenever the system receives an origin and a destination, it shall display the top 5 candidate Paths ranked by score.

[R28] If no fully bike-path-only route exists between the selected origin and destination, the system shall still display the map with all available bike-path StreetSegments highlighted, allowing the User to inspect partial bike-path options.

[R29] Upon request, the system shall show detailed information about a selected Path, including the sequence of StreetSegments, their status, obstacles, and the aggregated user contributions associated with them.

[R30] The system shall visually highlight on the map only those StreetSegments that are currently marked as bike paths, distinguishing them from regular street segments.

Personalized Route Recommendations

[R31] Whenever a Registered User requests a route, the system shall tailor the ranking of available paths according to the User's saved preferences.

[R32] Whenever path scoring is computed, the system shall consider:

- path condition,
- number and type of obstacles,
- consensus level from contributions,
- efficiency toward destination,
- traffic comfort indicators.

Data Aggregation and Consolidation

[R33] Whenever the aggregated BikePathSuitability for a StreetSegment no longer satisfies the suitability criteria for cycling (e.g., recent contributions reporting heavy traffic or unsafe speed), the system shall unmark that StreetSegment as a bike path and treat it as a regular street segment.

[R34] Whenever the bike-path status or aggregated condition of one or more StreetSegments changes, subsequent route computations shall use the updated information, so that Paths containing StreetSegments no longer suitable for cycling are no longer proposed among the best bike routes.

Weather Data Enrichment

[R35] Whenever an Automatic trip is saved, the system shall request weather information (temperature, wind speed, weather conditions) from an external service based on trip time and location.

[R36] Whenever weather data is retrieved, the system shall associate it with the corresponding trip summary.

2.4. User Characteristics

The BBP platform is designed for two main categories of users, each with different goals, levels of interaction, and technical proficiency.

- **Registered Users**
  Registered cyclists use the platform to record their trips, contribute information about bike paths, and access personalized features such as tailored route recommendations. They are typically familiar with mobile applications, GPS-based services, and standard interaction patterns such as registration, profile configuration, and map navigation. Their technical proficiency is generally medium to high, although the system is designed to remain accessible for less experienced users. Their motivation is to track their cycling activity, improve route planning, and contribute to the quality of shared path information. They may interact with the platform on a daily or weekly basis, especially when using BBP as part of their commuting routine or during periods of regular training.

- **Unregistered Users**
  Unregistered users access BBP to explore available bike paths without creating an account. Their interaction is limited to map browsing, viewing path details, and performing basic

searches between an origin and destination. These users vary widely in their technical abilities, ranging from casual cyclists with minimal experience using navigation apps to proficient riders looking for quick route information. Their engagement frequency is typically low, consisting of occasional use on weekends or when planning new itineraries. Since they cannot record trips or publish data, their expectations focus primarily on a simple, intuitive map interface and clear visualization of path conditions.

- **Cross-User Expectations**
  All users expect BBP to provide: a simple and intuitive interface, accessible even to those with limited technical skills; reliable and up-to-date information on path conditions, obstacles, and safety; fast and accurate map navigation and trip visualization; privacy-respecting handling of location and sensor data. The system accommodates diverse levels of cycling expertise and device familiarity, ensuring accessibility and usability across the entire user base.

2.5. Assumptions, Dependencies and Constraints

2.5.1. Domain Assumptions

[DA1] The urban cycling infrastructure can be accurately represented by a finite set of interconnected paths.

[DA2] Each path corresponds to a real, navigable route that cyclists can take within the target city or area.

[DA3] Users always interact with the system through a device with a reliable Internet connection.

[DA4] Users who sign up to the platform possess a valid and active email address.

[DA5] Users can access the mailbox of the email address provided during registration.

[DA6] Registered Users own a mobile device equipped with GPS, accelerometer, and gyroscope sensors.

[DA7] When automatic tracking is enabled, the mobile device's sensors operate correctly and provide accurate data samples.

[DA8] Users do not intentionally provide false information about bike paths, obstacles, or conditions.

[DA9] Obstacles detected by the sensors during a trip correspond to actual modifications or

irregularities of the physical path.

[DA10] The external meteorological services queried by BBP always return correct and timely information based on geographical coordinates and timestamps.

[DA11] Map data and geolocation services used by BBP are available and functioning when the system computes paths between origin and destination.

[DA12] Users who manually insert or edit path information correctly identify the streets and segments they intend to describe.

[DA13] Users who confirm or correct automatically detected obstacles provide honest and meaningful feedback, improving the reliability of aggregated contributions.

[DA14] The timestamps associated with trips and contributions reflect the actual moment in which the data was collected.

[DA15] When the system aggregates contributions, the recency and consistency rules are meaningful because users submit updates promptly after observing path changes.

3 | Specific Requirements

3.1. External Interface Requirements

3.1.1. User Interfaces

This section presents the main screens of the application through conceptual mock-ups, illustrating how users can navigate maps, manage their profiles, record trips, confirm detections, and insert manual reports. The goal is to provide a clear and intuitive overview of the graphical layout and interaction flows that support both registered and unregistered users in accessing the platform's services. In the following mock-ups, we showcase the functionalities available to a registered user, as this allows us to present a broader and more complete view of the application's capabilities.

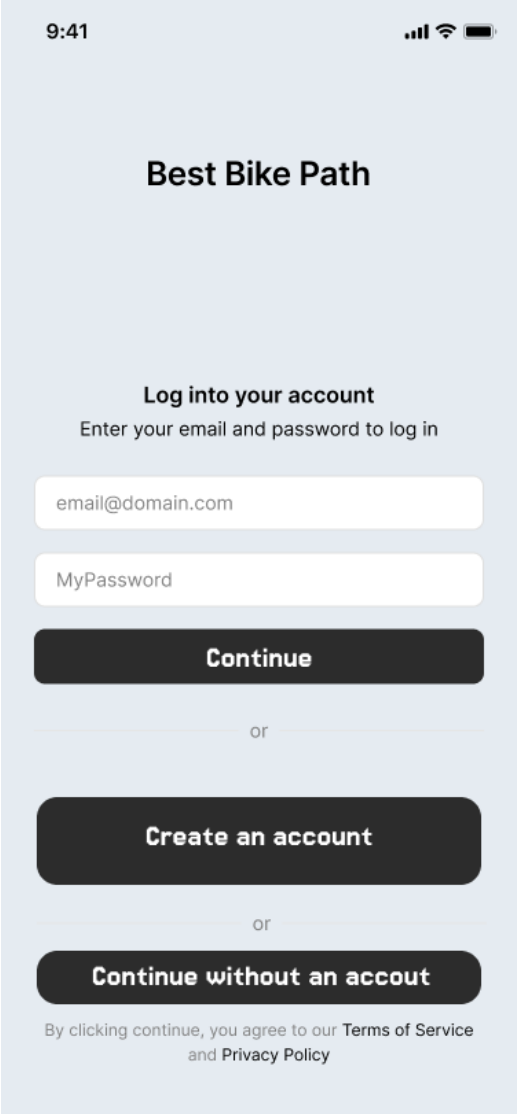


Figure 3.1: Log in Interface

Figure 3.1 represents the initial interface that any user encounters upon opening the application. From this screen, the user may choose to log in, sign up, or continue without creating an account, accessing a limited set of functionalities.

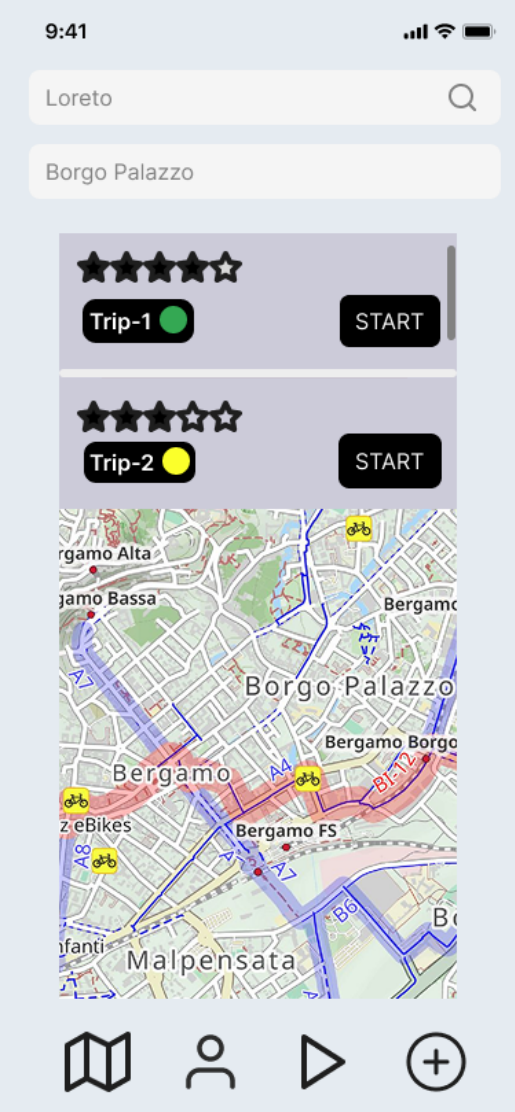


Figure 3.2: Map Interface

Figure 3.2 shows the map interface, which is displayed immediately after signing in or by selecting the map icon in the bottom navigation bar. At the top of the screen, a search bar allows the user to specify an origin and a destination in order to look for suitable cycling paths. Just below it, the suggested paths are presented, where the stars indicate the overall score of each path and the colored circle on the left represents the quality of the road. The map provides an overview of the available routes and serves as the central entry point for consulting path information and exploring the surrounding area.

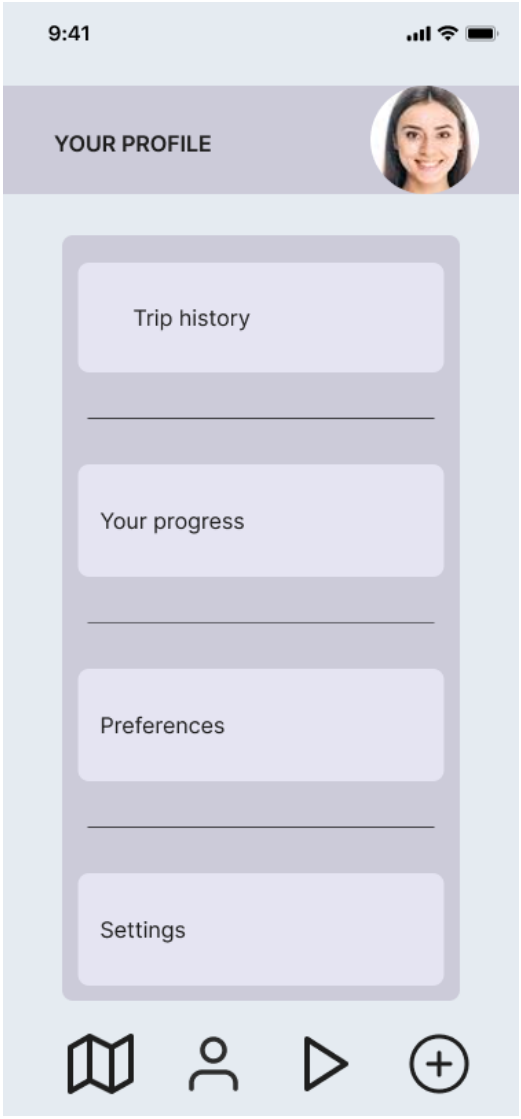


Figure 3.3: Profile interface

Figure 3.3 illustrates the profile interface, which can be accessed by selecting the human-shaped icon in the bottom navigation bar. The *Trip History* section displays all the user's completed trips and allows them to publish any unpublished paths as well as view detailed statistics for each ride. The *Your Progress* section summarizes the user's overall activity, including metrics such as the total number of completed trips and the cumulative distance traveled. In the *Preferences* section, the user can configure their preferred path qualities, enabling the system to provide personalized route suggestions. Finally, the *Settings* section allows the user to modify their account information, such as changing their profile picture or deleting their account.

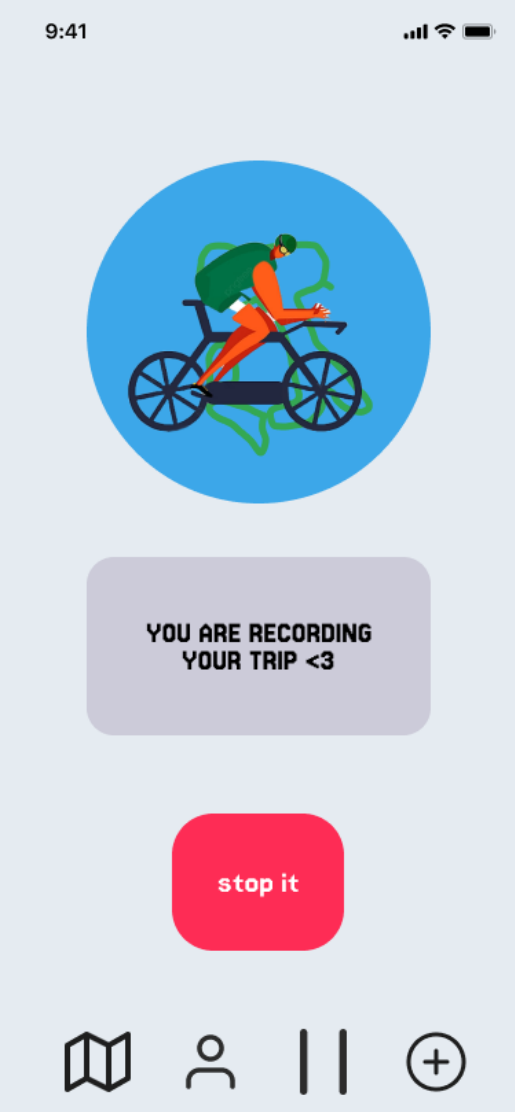


Figure 3.4: Automatic trip interface

Figure 3.4 shows the interface displayed after pressing the start button in the bottom navigation bar, which initiates the automatic trip detection mode. During this mode, the system records the user's movement and collects sensor data to reconstruct the traveled path. The trip can be terminated at any time either by pressing the *Stop* button on the screen or by selecting the same bottom navigation button, which now functions as a stop control. While the trip is being recorded, the user may continue to navigate and use the application freely without interrupting the ongoing detection process.

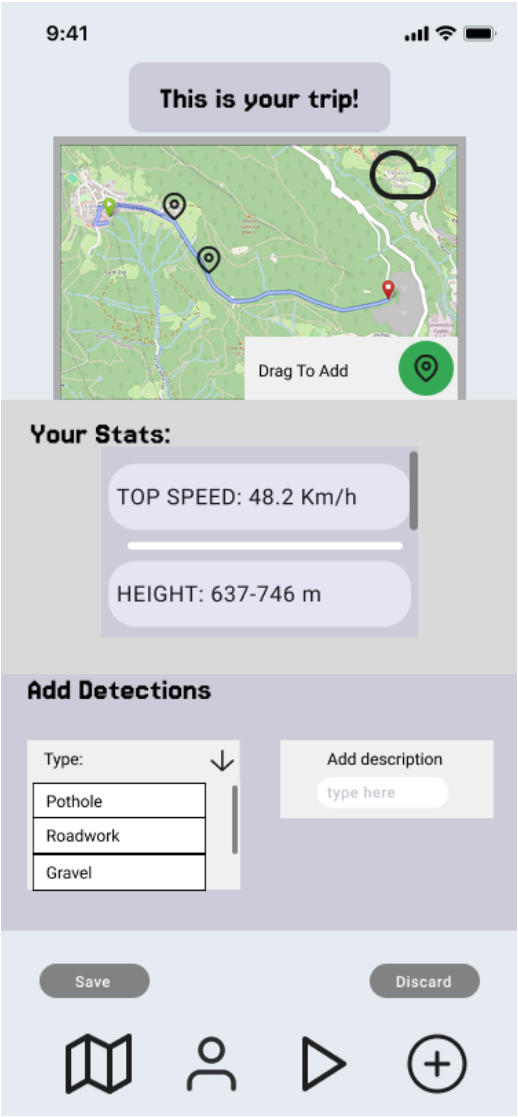


Figure 3.5: Detection confirmation interface

Figure 3.5 shows the interface displayed after stopping the automatic trip detection illustrated in Figure 3.4. In this screen, the user can review the statistics of the completed trip and examine the automatically detected events. The lower section allows the user to add new detections or modify the ones suggested by the system. To precisely indicate where a detection occurred, the user can drag the pin on the map, positioning it on the exact road segment where the event took place.



Figure 3.6: Manual trip interface

Figure 3.6 can be accessed by pressing the "+" icon in the bottom navigation bar, allowing the user to create a trip manually. In this interface, the user may build a route by entering the names of the streets or by selecting their beginning and end points by clicking directly on the map and choosing an existing path between the two points. The blue lines represent the selected street segments. Once the route has been created, the user may continue forward to add the detections by pressing the *Confirm path* button. In the following screen after having added the desired detections the user has the ability to either save the trip in his history or to directly post it as a contribution without saving it in his account.

3.1.2. Hardware Interfaces

The system shall ensure full compatibility with the hardware components involved in the interaction between users' devices and the platform. The following requirements apply:

User Devices

- The platform shall support access from a wide range of commonly used device such as tablet sand smartphones.
- The platform shall be fully usable across different screen sizes and resolutions, ensuring responsiveness and readability regardless of device type.
- User devices shall include sufficient storage and processing capabilities to runt he mobile application, including support for caching data and executing client-side scripts.
- For devices using automatic trip detection or sensor-based functionalities, the system shall interface with hardware components such as **GPS receivers**, **accelerometers**, and **gyroscopes**. These components must be accessible to the application for acquiring motion data, trip-related information, and geolocation.

3.1.3. Software Interfaces

The system is designed as a mobile application, capable of performing all its core functionalities without relying on essential third-party software. Nevertheless, the system shall comply with the following software interface requirements:

- The system shall function correctly across the most widely adopted operating systems, including, but not limited to, **Android**, and **iOS**.
- The system shall interface with geolocation and mapping services to retrieve, compute, and display cycling routes. These services include GPS modules, map-rendering libraries, and geocoding/route-search components.
- The system shall interface with external meteorological information providers to retrieve weather conditions relevant to trips and route safety evaluations.
- The system shall interact with mobile device sensors, including accelerometer and gyroscope, to collect motion-related data supporting the automatic detection of obstacles and road conditions.
- No additional software installations or configurations shall be required on the user's device, aside from using the official mobile application.

3.1.4. Communication Interfaces

The system shall support reliable and secure communication mechanisms enabling interaction between client devices, backend services, and external data providers. The following requirements apply:

- The system shall operate over standard Internet connections, including both wired and wireless networks such as **Ethernet**, **Wi-Fi**, and **mobile data**.
- All communication between the client and the server shall be secured through **encrypted protocols** (e.g., TLS/SSL) to protect sensitive information such as user credentials, personal data, recorded trips, and sensor-based contributions.
- The system shall utilize widely adopted **web communication protocols**, primarily **HTTPS**, to ensure secure and reliable data transmission between user devices and the backend.
- The system shall support communication with external services, including meteorological data providers and geolocation/map services, through standard **HTTP/HTTPS-based APIs**.
- No additional software installations or communication packages shall be required on the user's device to enable proper communication with the platform.
- The system shall minimize communication latency by adhering to industry-standard response times ensuring a smooth and responsive user experience.

Hosting Infrastructure

- The system shall be deployed on a hosting infrastructure capable of supporting simultaneous access by multiple users with minimal latency.
- The hosting environment shall provide sufficient computational resources for processing real-time map data, aggregating user contributions, and executing background operations.
- The infrastructure shall include adequate storage capabilities to manage user profiles, trips, sensor data, and historical records of contributions.
- The hosting environment shall be scalable, allowing the system to accommodate increasing traffic, data volume, and computational demands over time while preserving service availability and performance.

3.2. Functional Requirements

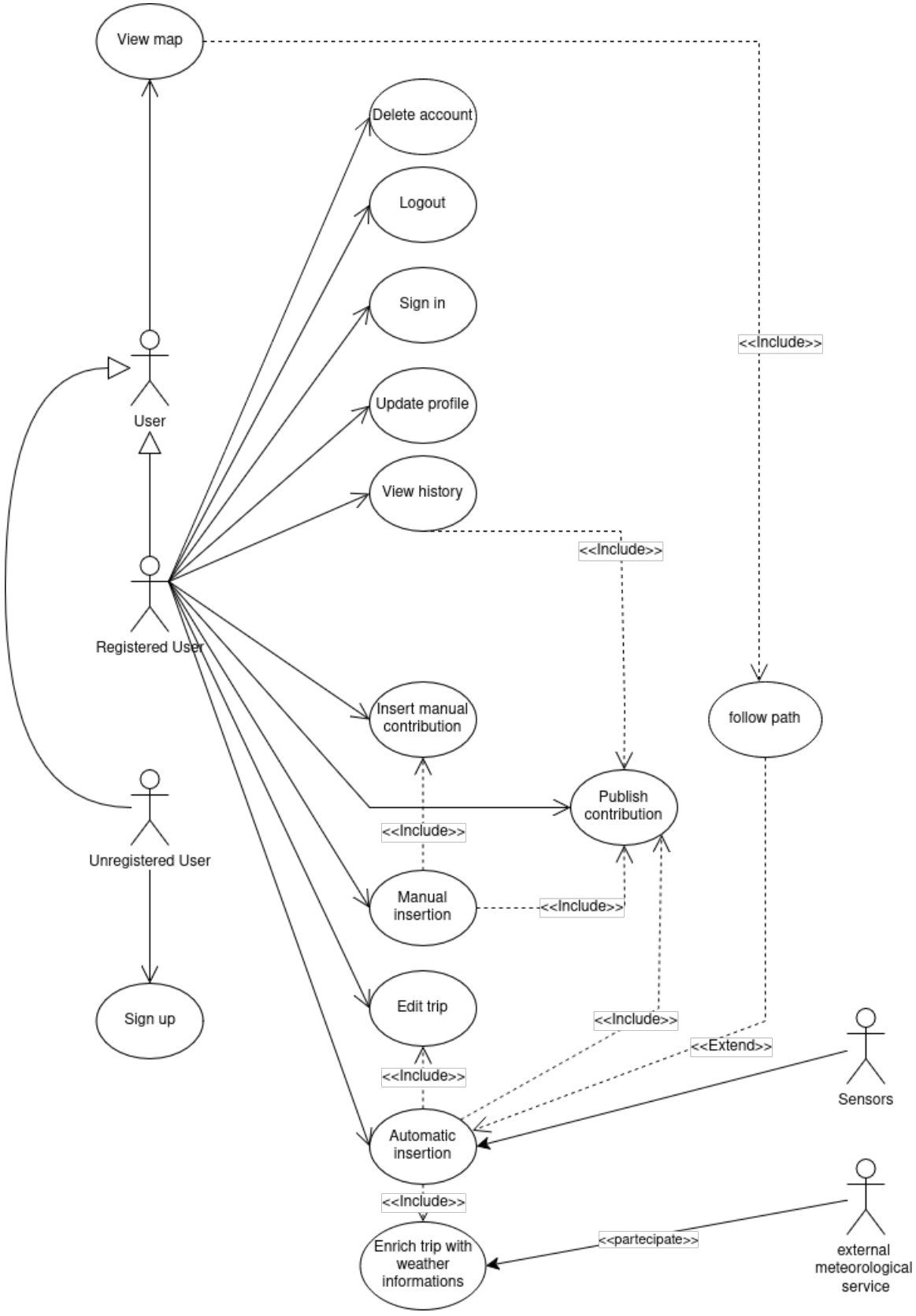


Figure 3.7: Use Case Diagram

3.2.1. Use Cases and Activity Diagrams

UC1. Sign Up

Actor	User
Entry Conditions	The User is not logged into the platform and is on the homepage.
Flow of Events	1. On the homepage, the User clicks the "Sign up" button. 2. The system displays the registration form. 3. The User inserts the required registration details (e.g., email, password, password confirmation and role). 4. The User confirms the provided information by clicking the "Register" button. 5. The system checks the completeness and validity of the submitted data. 6. If the data is valid, the system sends a confirmation email containing a verification link that expires after 24 hours. 7. The User clicks the confirmation link in the email before it expires. 8. The system allows the User to access the profile page and complete the profile by inserting the remaining personal details.
Exit Conditions	The User account is created and verified, and the profile is completed so that the User can access the platform functionalities.
Exceptions	• Some mandatory fields are missing: the system notifies "Missing mandatory fields" and does not allow the User to proceed until all required fields are filled in. • The password does not satisfy the platform's security requirements: the system notifies "Invalid password" and prompts the User to correct it. • The email address is already linked to an existing account: the system notifies "Email already linked" and redirects the User to the login page. • The confirmation link expires before being used: the system discards all data previously inserted by the User and invalidates the link.

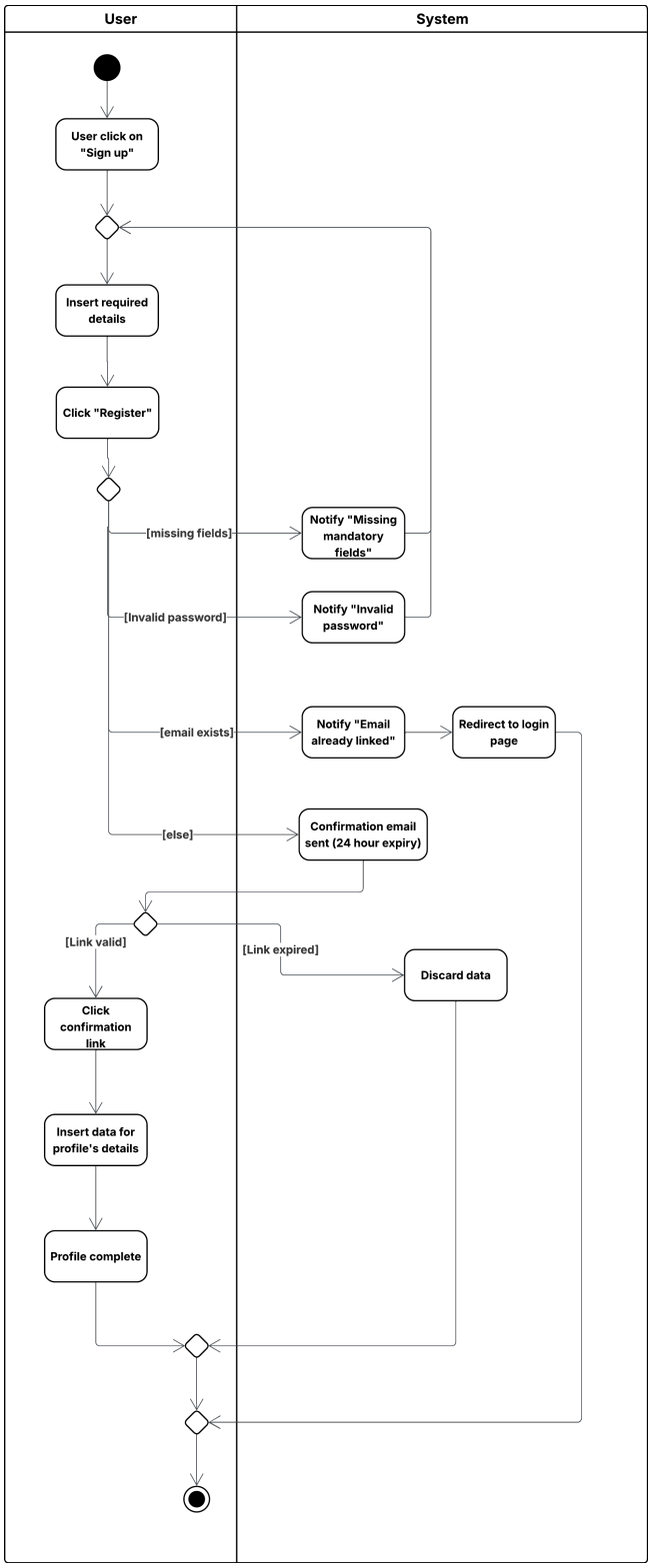


Figure 3.8: Sign Up

UC2. Sign In

Actor	User
Entry Conditions	The User is not logged into the platform and is on the homepage.
Flow of Events	1. On the homepage, the User clicks the "Sign in" button. 2. The system displays the login form. 3. The User enters their credentials (email and password). 4. The User clicks the "Login" button to submit the credentials. 5. The system validates the provided credentials. 6. If the credentials are valid and the account is verified, the system checks whether the User's profile is complete. 7. If the profile is complete, the system redirects the User to the Home page. 8. If the profile is not complete, the system redirects the User to the "Update profile" page.
Exit Conditions	The User is successfully authenticated and is redirected either to the Home page or to the profile completion page.
Exceptions	• The entered credentials are not valid: the system notifies "Entered credentials are not valid" and does not log the User in. • The account associated with the provided email is not verified: the system notifies "Account not verified" and does not log the User in.

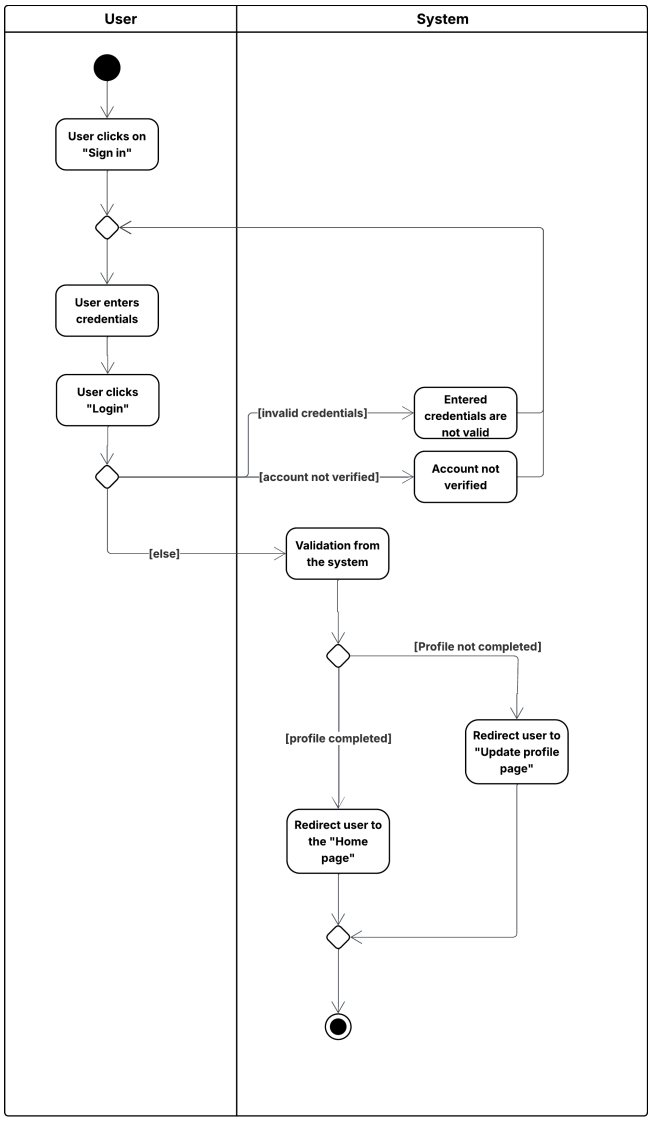


Figure 3.9: Sign In

UC3. Delete Account

Actor	Registered User
Entry Conditions	The Registered User is logged into the platform and is on the account settings page.
Flow of Events	1. The User opens the account settings section. 2. The User selects the "Delete account" option. 3. The system shows a warning explaining the consequences of account deletion and asks the User whether they want to continue. 4. The User confirms the intention to proceed with account deletion. 5. The system requests re-authentication by asking the User to enter their credentials. 6. The User inserts their credentials and submits them. 7. The system validates the provided credentials. 8. If the credentials are valid, the system removes the User profile and related personal information. 9. The system deletes the trips stored for the User. 10. The system deletes the User's preferences and local settings. 11. The system invalidates any active sessions and authentication tokens associated with the User. 12. The system notifies the User that the account has been successfully deleted.
Exit Conditions	The User account and all associated data are deleted, and the User is no longer able to access the platform with the previous credentials.
Exceptions	• The User cancels the deletion after viewing the warning: the system returns the User to the account settings page without performing any deletion. • The re-authentication credentials are not valid: the system notifies the User that the credentials are invalid and returns to the account settings page without deleting the account.

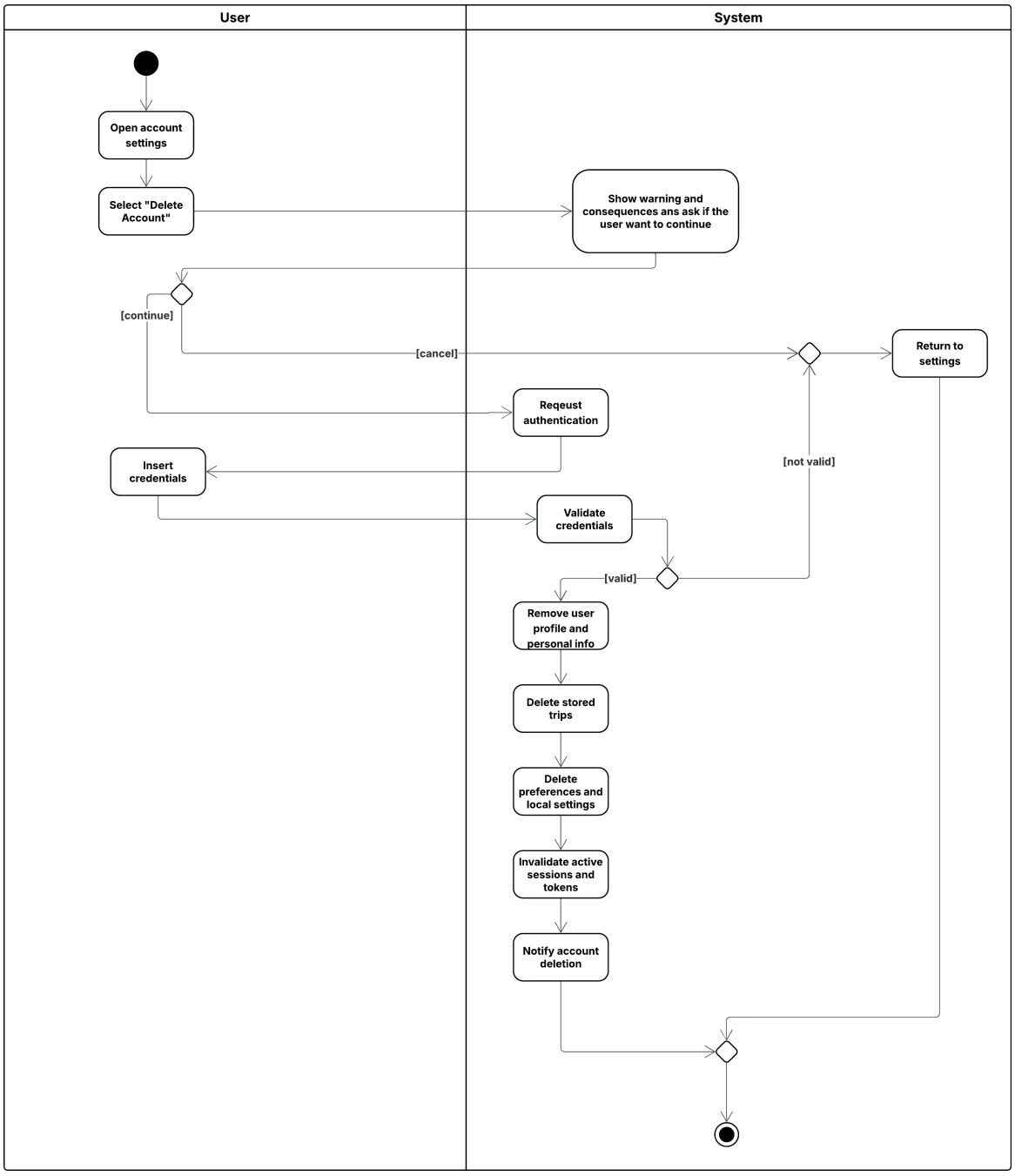


Figure 3.10: Delete Account

UC4. Logout

Actor	User
Entry Conditions	The User is logged into the platform.
Flow of Events	1. The User clicks on the "Logout" button. 2. The system receives the logout request. 3. The system terminates the User's session and invalidates any active authentication token. 4. The system redirects the User to the Home screen.
Exit Conditions	The User's session is terminated, and the User is no longer authenticated on the platform.
Exceptions	• None.

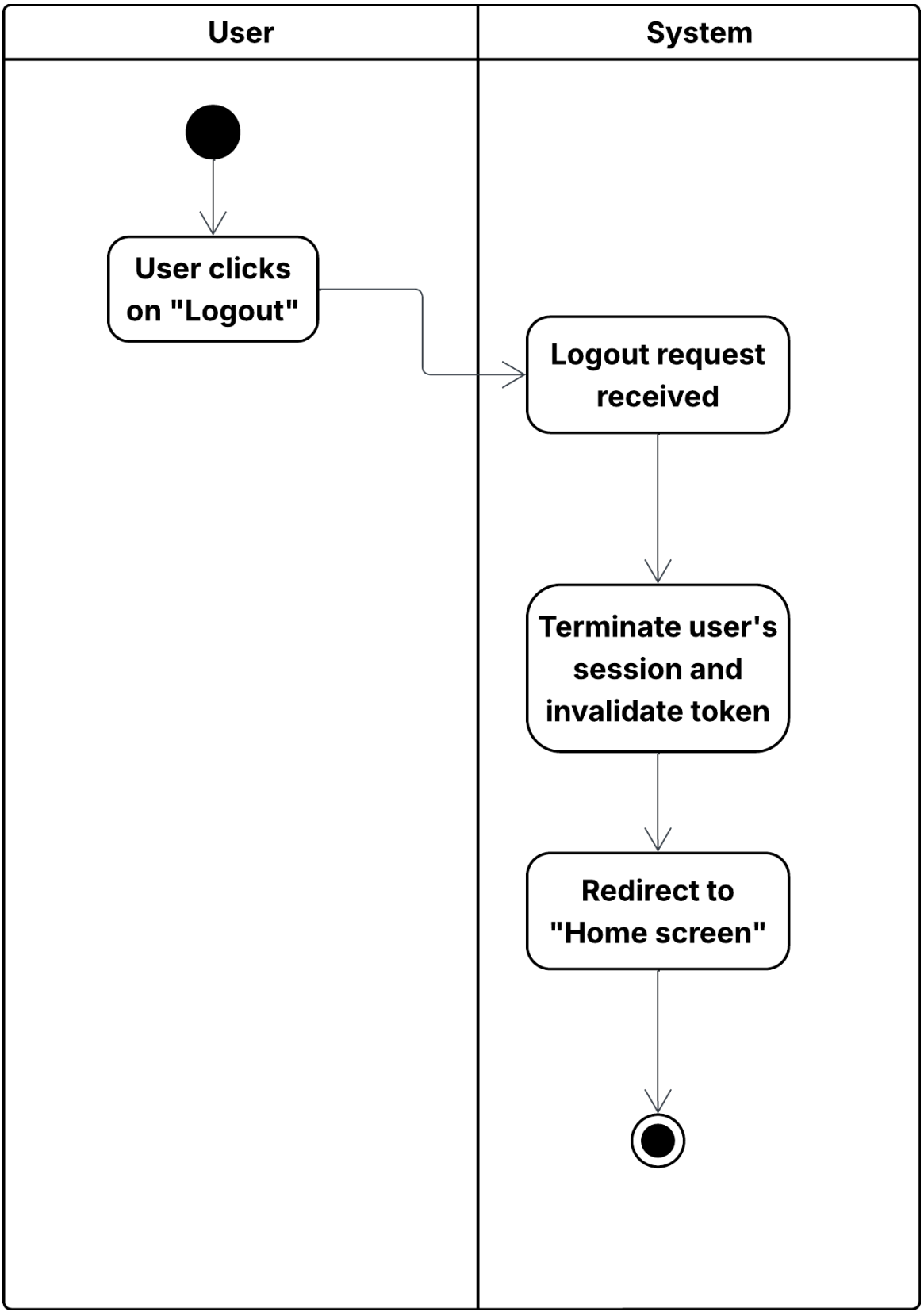


Figure 3.11: Logout

UC5. Edit Profile

Actor	Registered User
Entry Conditions	The Registered User is logged into the platform.
Flow of Events	1. The User clicks on the "Profile" section. 2. The User clicks on the "Edit profile" option inside the settings. 3. The system displays the editable profile form, including personal data and preference fields. 4. The User updates the desired information, such as personal details and visualization preferences (e.g., preferred path filters based on past trips). 5. The User clicks the "Apply" button to confirm the changes. 6. The system validates the submitted changes. 7. If the changes are valid, the system stores them and notifies the User of the successful update.
Exit Conditions	The User's profile and preferences are updated and consistently stored in the system.
Exceptions	• Some updated fields are not valid (e.g., invalid email format or unsupported preference values): the system notifies the User that the changes are not valid and does not apply them.

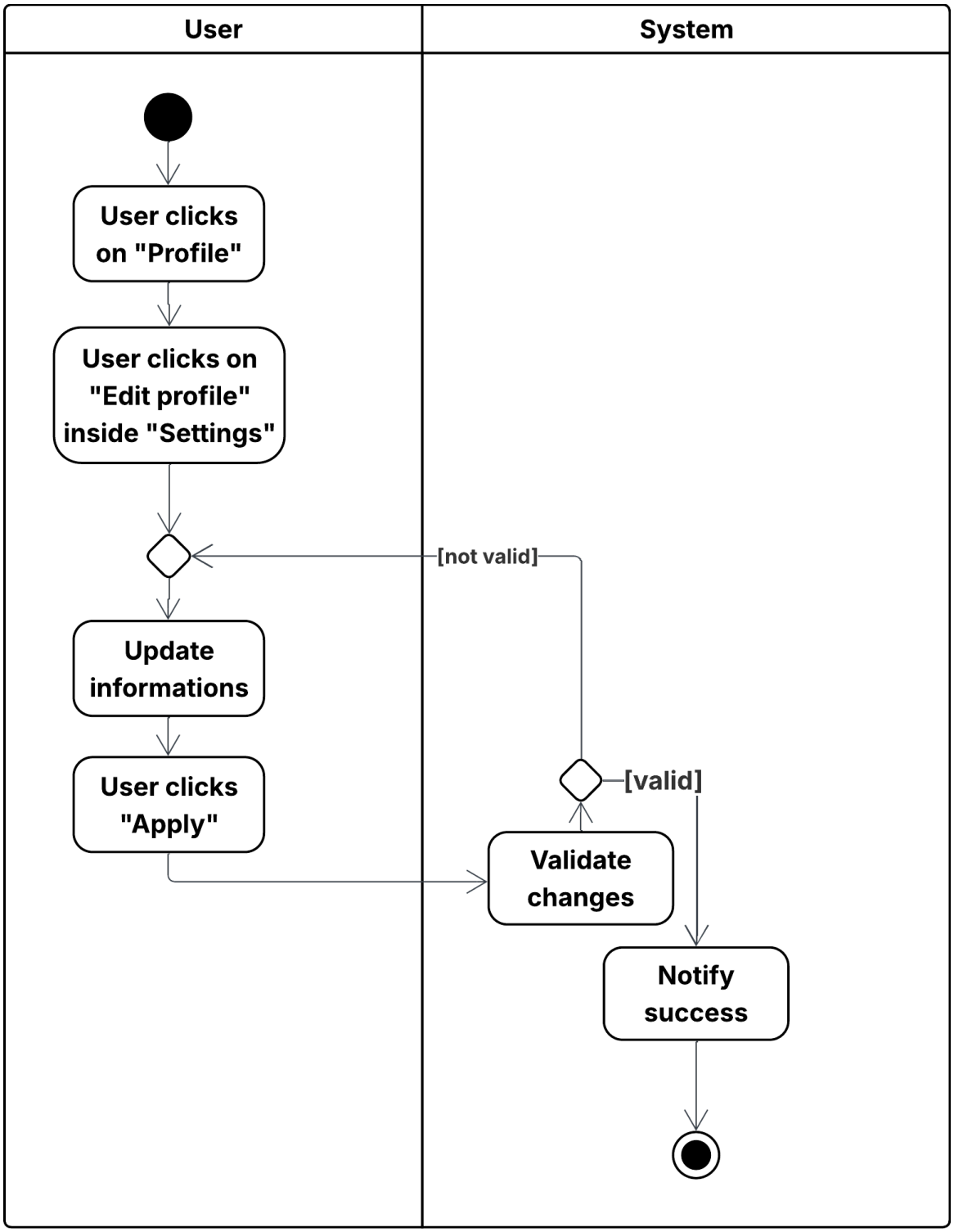


Figure 3.12: Edit Profile

UC6. View Trip History

Actor	Registered User
Entry Conditions	The Registered User is logged into the platform.
Flow of Events	1. The User clicks on the "My trips" section. 2. The system retrieves the trips associated with the User and shows the list of available trips. 3. The User selects one trip from the list. 4. The system shows the selected trip together with its stored data and associated contributions. 5. The User chooses an action for the selected trip (e.g., to publish its contributions or to simply inspect the data). 6. If the User decides to publish the contributions of the trip, the system starts the UC. Publishing Contribution use case for the first contribution associated with that trip. 7. The system iteratively processes the contributions linked to the trip, publishing them one at a time so that merge and update policies on paths can be correctly applied for each contribution. 8. When there are no more contributions to process, the system returns control to the trip view. 9. For readability, the activity diagram omits the fact that the User can navigate back to previous pages (trip list or other sections of the app) at any point in the interaction.
Exit Conditions	The User has viewed the selected trip and, if desired, has published some or all of the contributions associated with that trip.
Exceptions	• The User has no stored trips: the system shows an empty list and may display an informative message. • The selected trip has no associated contributions: the system shows the trip data but does not start the publishing process.

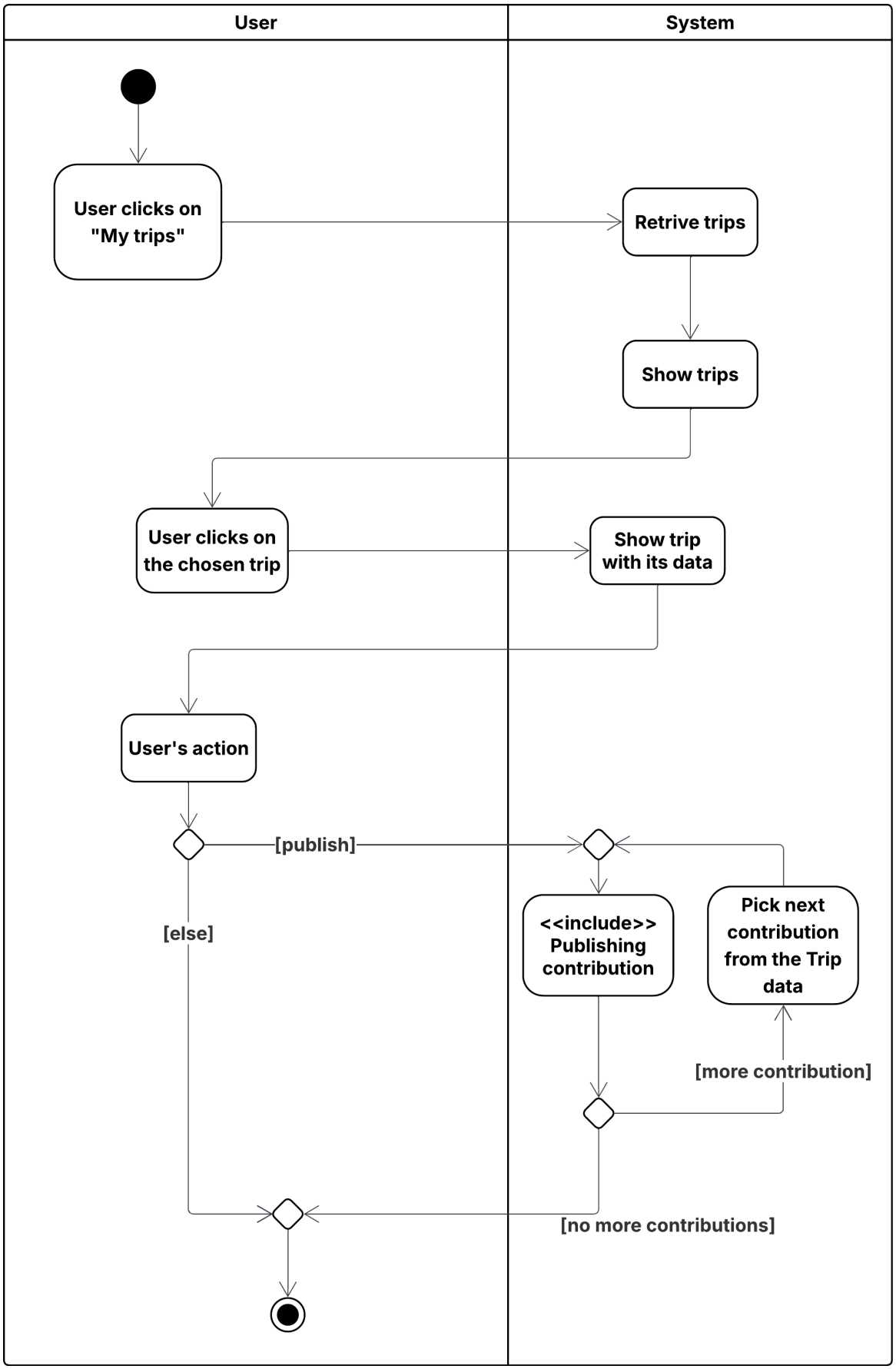


Figure 3.13: View Trip History

UC7. Publish Contribution

Actor	System (invoked from other use cases, e.g. View Trip History)
Entry Conditions	A Contribution has been selected for publication (for example, from a Trip in the User's history).
Flow of Events	1. The system validates the selected Contribution, checking that its data is complete and consistent. 2. If the Contribution is valid, the system retrieves from the database any existing Contribution(s) referring to the same street or path segment. 3. The system applies the configured merging policy to combine the new Contribution with the existing information (e.g., updating or aggregating obstacle data on that path). 4. The system updates the corresponding path information in the database according to the result of the merging policy. 5. The system flags the User's Contribution as published. 6. The system notifies the successful publication of the Contribution to the invoking use case.
Exit Conditions	The Contribution is marked as published and the affected path information in the database has been updated according to the merging policy.
Exceptions	• The Contribution is not valid (e.g., missing mandatory data or inconsistent location): the system notifies the failure and does not modify any existing path information. • A database error occurs while retrieving or updating existing Contributions: the system notifies the failure and leaves the User's Contribution in a non-published state.

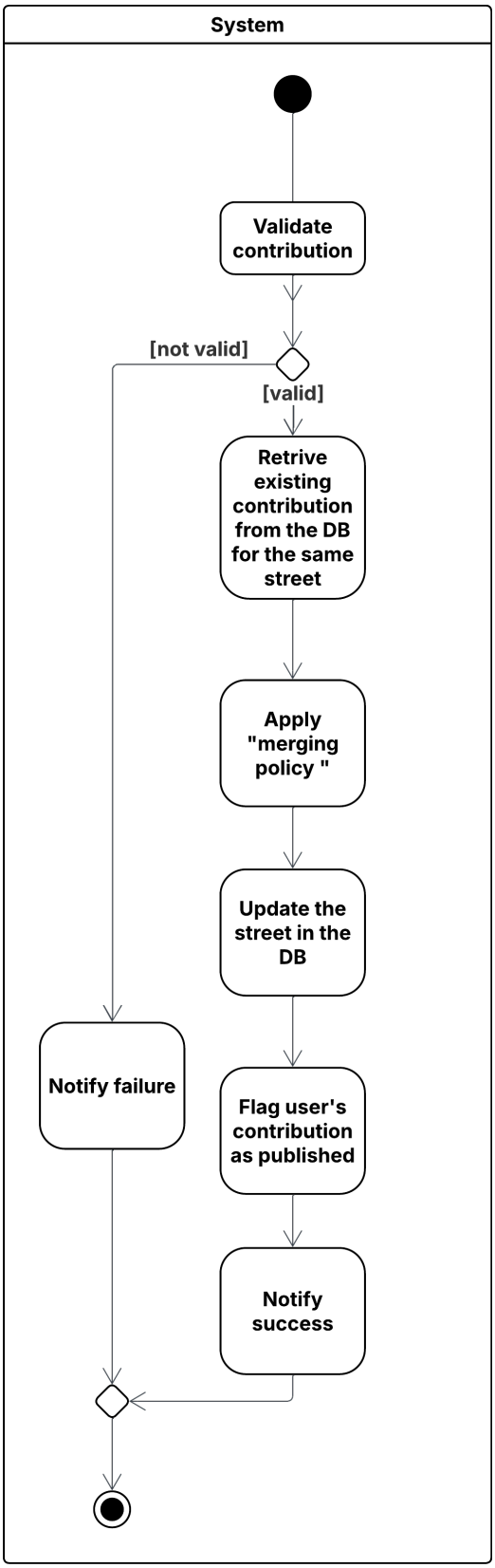


Figure 3.14: Publish Contribution

UC8. View Map

Actor	User
Entry Conditions	The User is logged into the platform and has access to the map functionality.
Flow of Events	1. The User clicks on the "Map" section. 2. The system loads the default map region (e.g., around the User's current position or last viewed area). 3. The system retrieves the visible bike-path data for the current map region. 4. The system displays the map together with the visible paths and their associated contributions (e.g., obstacles, warnings, roadworks). 5. The User chooses an action on the map view (e.g., search for a path, tap on a displayed path segment, move around the map, or start following a path). 6. If the User searches for a specific path or destination, the system invokes the UC. Search Path use case and, after the search is completed, updates the map with the best suggested paths (e.g., top 5). 7. If the User taps on a displayed path, the system shows the selected path with more detailed information, including aggregated contributions along that path. 8. If the User decides to follow a selected path, the system redirects the User to the "Automatic trip mode" page and invokes the UC. Follow Path use case. 9. If the User simply moves around the map (pan/zoom), the system retrieves bike-path data for the newly visible region and refreshes the map with updated paths and contributions. 10. For readability, the activity diagram omits that the User can exit the map or return to previous sections of the application at any time during the interaction.
Exit Conditions	The User has visualized the desired map area, inspected the available paths and related contributions, and optionally started following a selected path.

Exceptions	• No bike-path data is available for the current map region: the system still displays the map but shows an informative message indicating that no paths are present. • A temporary error occurs while retrieving path data or contributions: the system notifies the User of the failure and suggests retrying the operation.

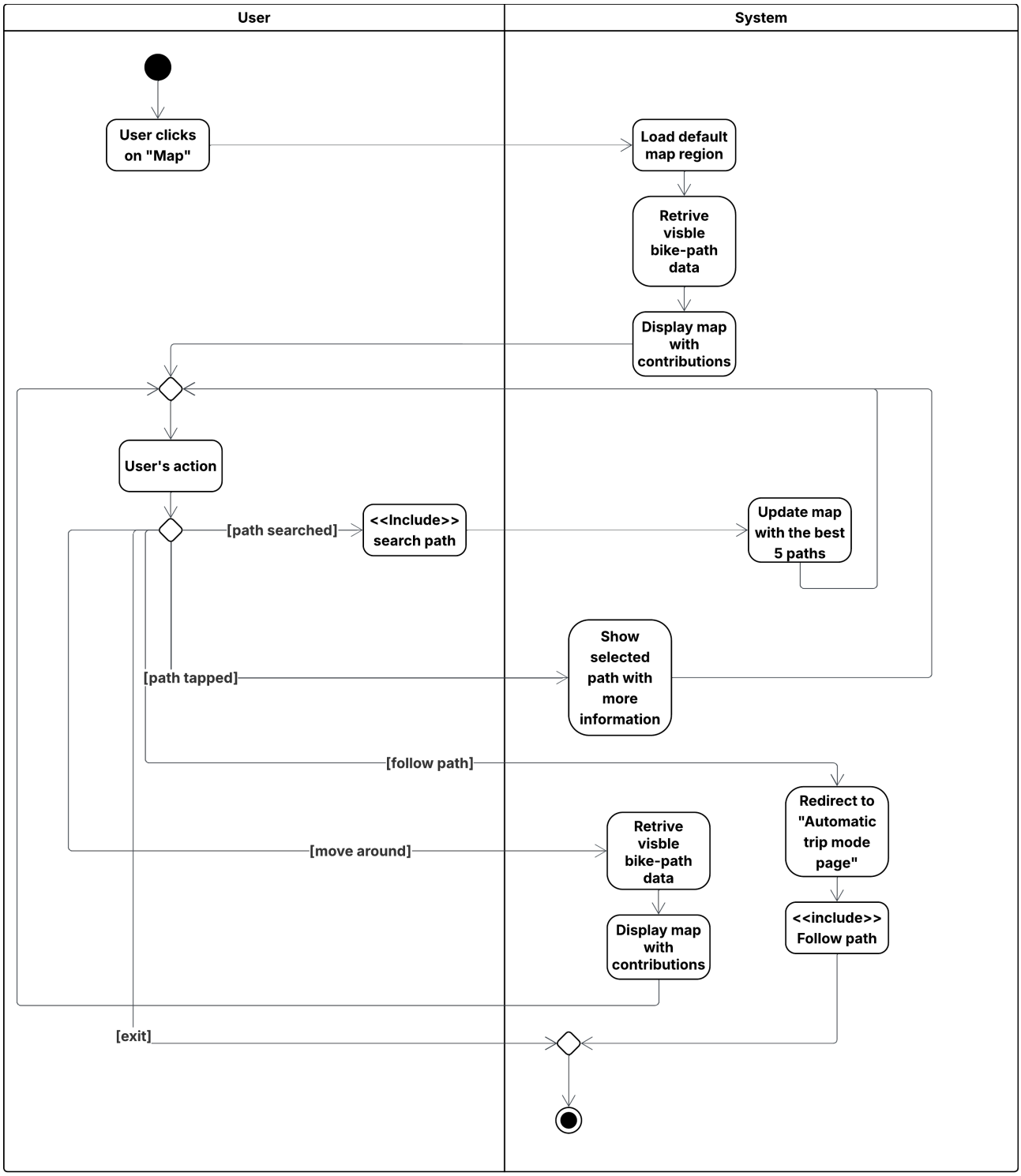


Figure 3.15: View Map

UC9. Search Path

Actor	User
Entry Conditions	The User is on a page where the map and the path search functionality are available (e.g., from UC. View Map).
Flow of Events	1. The User specifies the start and end points of the desired path (e.g., origin and destination on the map or via textual input). 2. The system retrieves from the database the bike paths that connect the specified start and end points, if any exist. 3. The system checks whether the User is registered or unregistered. 4. If the User is registered, the system orders the retrieved paths according to the User's saved preferences and the trips recently completed by the User (e.g., preferred difficulty, usual routes). 5. The system returns the list of searched bike paths to the invoking use case, already ordered if the User is registered.
Exit Conditions	A set of bike paths matching the specified start and end points is produced and made available to the calling functionality.
Exceptions	• No bike path exists connecting the specified start and end points: the system returns an empty result and may provide an informative message to the User. • A temporary error occurs while retrieving paths from the database: the system notifies the failure and suggests the User to retry the search.

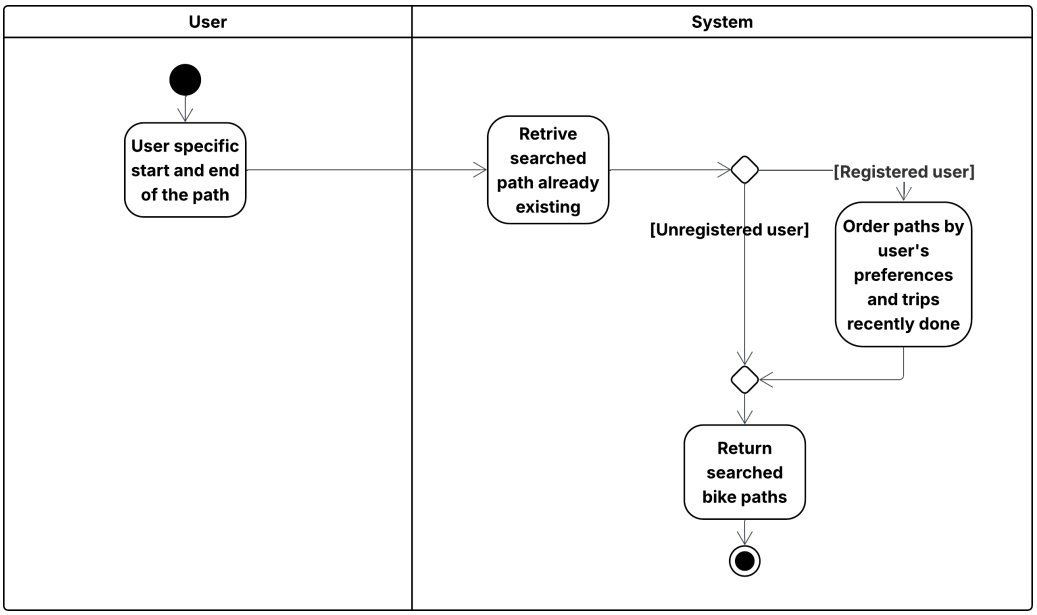


Figure 3.16: Search Path

UC10. Manual Insertion

Actor	Registered User
Entry Conditions	The Registered User is logged into the platform and has access to the manual trip insertion functionality.
Flow of Events	1. The system loads the default map region and initializes an empty *TripDraft*. 2. The User chooses how to add bike paths to the TripDraft (e.g., by manually selecting streets on the map or by searching for existing paths). 3. If the User chooses to search for a path, the system invokes UC. Search Path and lets the User select one of the returned paths. 4. For each selected or manually specified street, the system adds the corresponding bike path to the TripDraft and updates the map accordingly. 5. The system asks the User whether the definition of the trip is finished; the User may repeat the previous step to add more streets. For readability, the activity diagram does not explicitly show that the User can also remove or adjust previously selected streets. 6. Once the trip definition is complete, the system checks the validity of the TripDraft (e.g., at least one street, consistent geometry). If the TripDraft is not valid, the system shows an error and asks the User to correct it. 7. The User chooses an action on the completed TripDraft (e.g., add contributions, save as trip, publish as report, or discard). 8. If the User decides to add contributions, the User selects a street from the TripDraft and specifies a Contribution describing the status of that street (e.g., potholes, obstacles, roadworks). 9. The system checks the validity of each added Contribution and, if a Contribution is not valid, removes it from the TripDraft and notifies the User. 10. If the User chooses to save the TripDraft as a trip, the system saves the trip in the User's history and asks whether the associated Contributions should also be published (following the same iterative publication mechanism described for trips).

	11. If the User chooses to publish the information only as a report, the system treats the selected street and its Contribution as a standalone report, publishes the Contribution (without saving the TripDraft as a trip), and updates the corresponding path information. 12. If the User chooses not to publish any Contribution, the system simply saves the trip in the User's history without publishing. 13. If the User decides to discard the TripDraft, the system deletes all temporary data and terminates the use case.
Exit Conditions	The User has either saved a manually defined trip (optionally with published Contributions), published one or more standalone reports about specific streets, or discarded the manual insertion.
Exceptions	• The TripDraft is not valid (e.g., no street selected or inconsistent data): the system notifies the error and does not allow the User to save or publish until the trip is corrected. • One or more Contributions added to the TripDraft are not valid (e.g., missing mandatory fields): the system removes the invalid Contributions and asks the User to review them. • A temporary error occurs while saving the trip or publishing Contributions: the system notifies the failure and suggests the User to retry the operation.

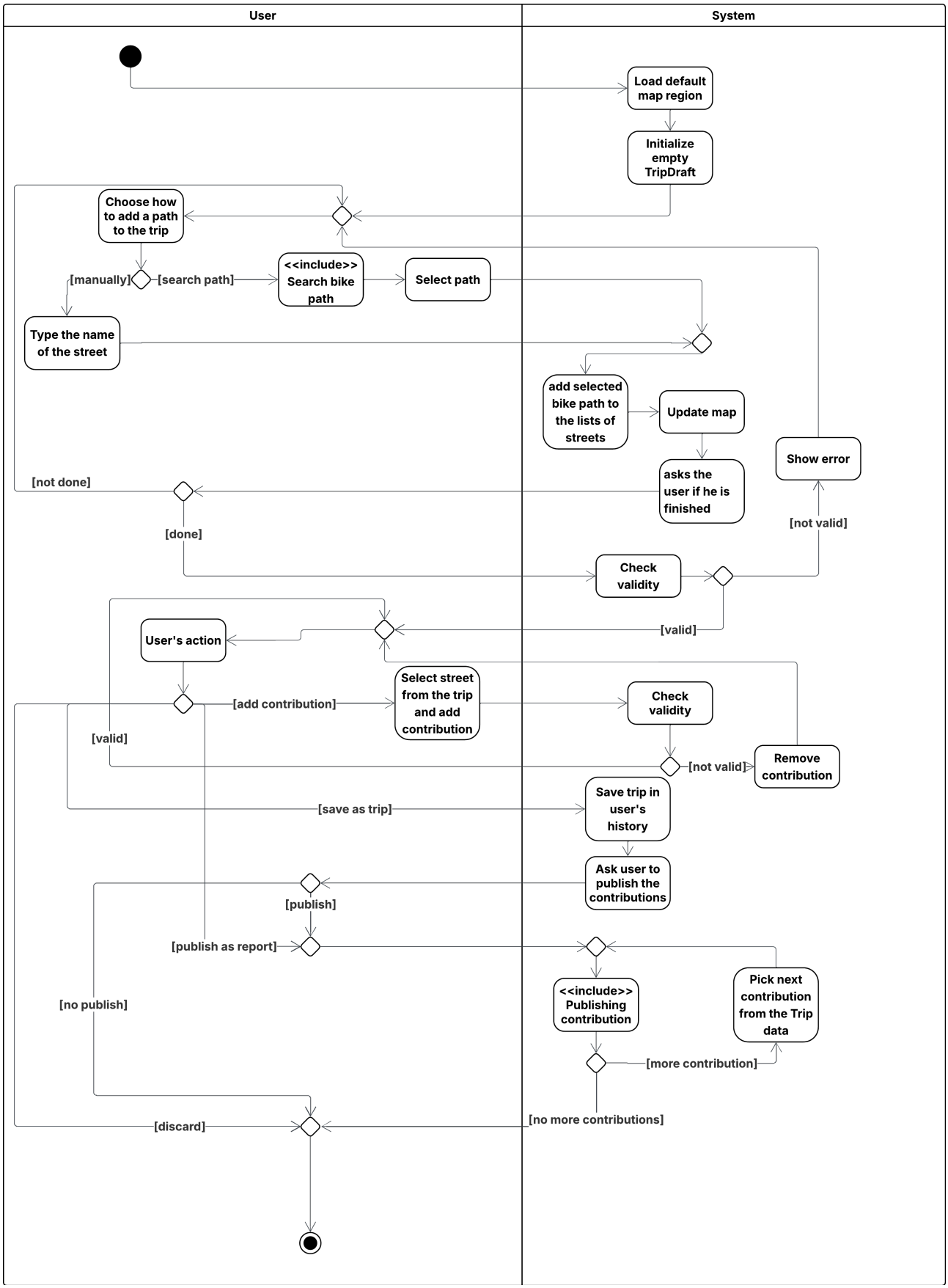


Figure 3.17: Manual Insertion

UC11. Automatic Insertion

Actor	Registered User
Entry Conditions	The Registered User is logged into the platform and has a device with GPS and motion sensors enabled.
Flow of Events	1. The User presses the "Start recording" button to begin an automatic trip. 2. The system initializes a new *RecordingSession* on the Client device. 3. The system starts GPS tracking and activates accelerometer and gyroscope monitoring. 4. While the recording is active, the system periodically acquires GPS samples and motion sensor data, storing them locally as raw trip data. 5. In parallel, the system monitors the User's movement to detect interruptions: consecutive GPS positions are compared and, if the distance travelled remains below a configurable threshold for a given time window, the User is marked as stationary; when movement resumes, the stationary status is cleared. 6. This logic, represented in the activity diagram, allows the system to identify stops and resumptions during the trip without performing semantic inference on the collected sensor data. 7. The recording continues until the system receives an "End recording" event from the User. 8. When the recording is stopped, the system stops GPS and motion sensors and finalizes the collection of raw trip data. 9. The system reconstructs the full path of the trip from the collected GPS trace. 10. Immediately after the recording phase, the system performs an inference process on the collected sensor data in order to generate a set of *draft Contributions* associated with the reconstructed path. These Contributions are not published nor merged at this stage and are intended solely for User review. 11. The system invokes the Data Enrichment Service, which computes aggregated trip statistics (e.g., distance, duration, speed profiles) and enriches the trip with external contextual information, such as weather data associated with the GPS trace, as described in UC. Enrich Trip with Weather Information.

	12. The system redirects the User to the "Review and edit" screen and includes UC. Edit Trip, allowing the User to inspect the automatically generated trip, modify its path if necessary, and edit, remove, or add Contributions before making any decision about saving or publishing the trip.
Exit Conditions	An automatic trip has been recorded with its reconstructed path, a set of draft Contributions generated through post-processing inference, and enriched contextual and statistical information; control is transferred to the manual review and editing phase.
Exceptions	• GPS or motion sensors are not available or cannot be started: the system notifies the User and does not start the automatic recording. • A temporary error occurs while acquiring sensor data: the system attempts to continue recording with partial data and informs the User that some portions of the trip may be incomplete.

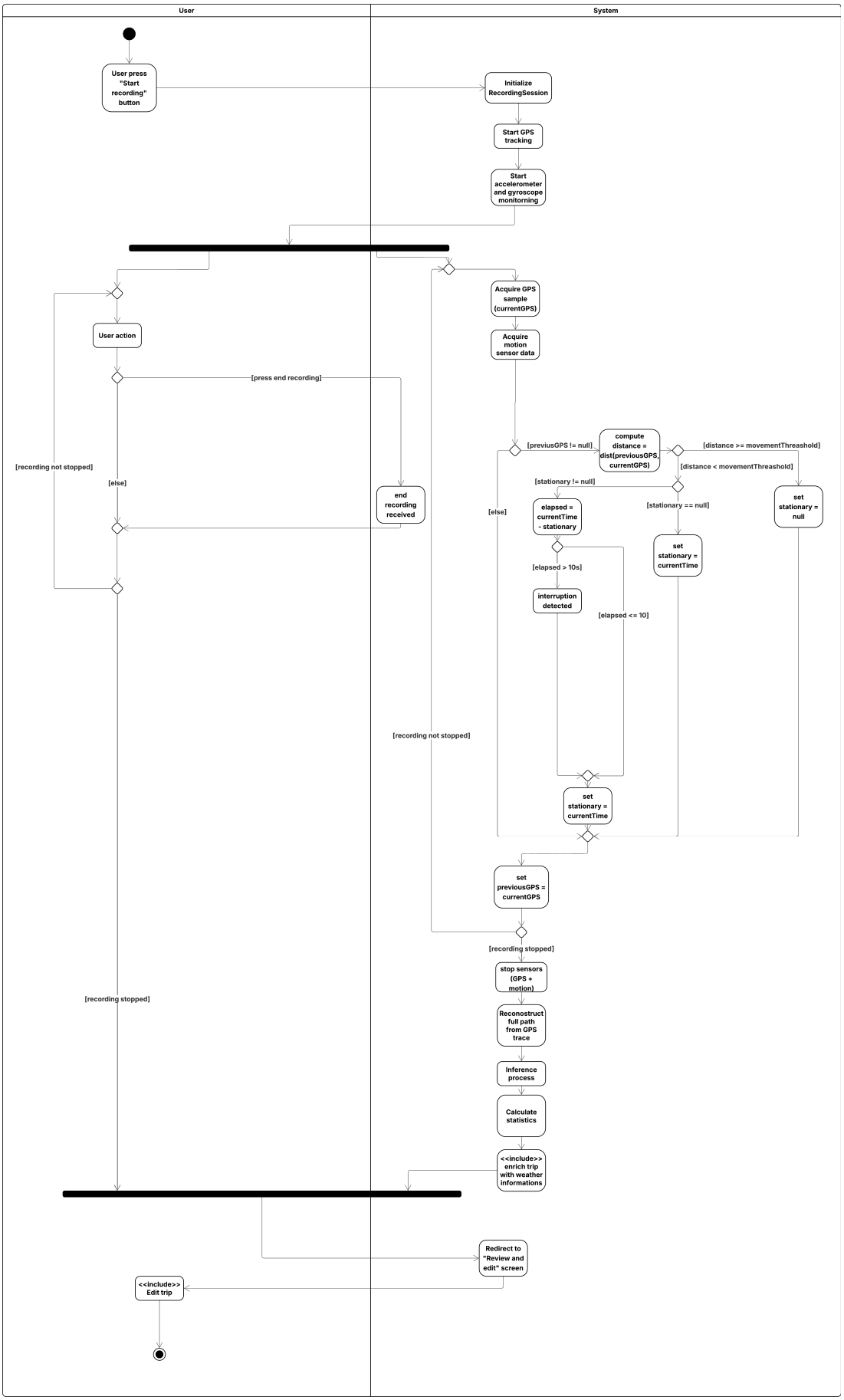


Figure 3.18: Automatic Insertion

UC12. Enrich Trip with Weather Information

Actor	System (Weather Service as external actor)
Entry Conditions	A TripDraft has been created and its GPS trace with timestamps is available.
Flow of Events	1. The system prepares a weather request using the trip's GPS trace and associated timestamps (e.g., start/end time and representative points along the route). 2. The system sends the request to the external Weather Service API. 3. The Weather Service receives the weather request. 4. The Weather Service checks its availability for the given time and location. 5. If the service is available, the Weather Service fetches the weather conditions corresponding to the requested time/place and returns the weather data to the system. 6. If the service is not available or an internal error occurs, the Weather Service returns an error response to the system instead of weather data. 7. Upon receiving the response, the system checks whether the request completed without errors. 8. If no error occurred, the system attaches the retrieved weather data to the TripDraft so that it can be stored together with the trip and shown in the trip details. 9. If an error occurred, the system marks the trip as "no weather data available" and continues without weather enrichment.
Exit Conditions	The TripDraft is enriched with weather information associated with its GPS trace, or it is explicitly marked as having no weather data available.
Exceptions	• The Weather Service is temporarily unavailable or returns an error: the system marks the trip as "no weather data available" and does not retry immediately. • The GPS trace is incomplete or does not contain valid timestamps: the system does not send the request to the Weather Service and leaves the TripDraft without weather information.

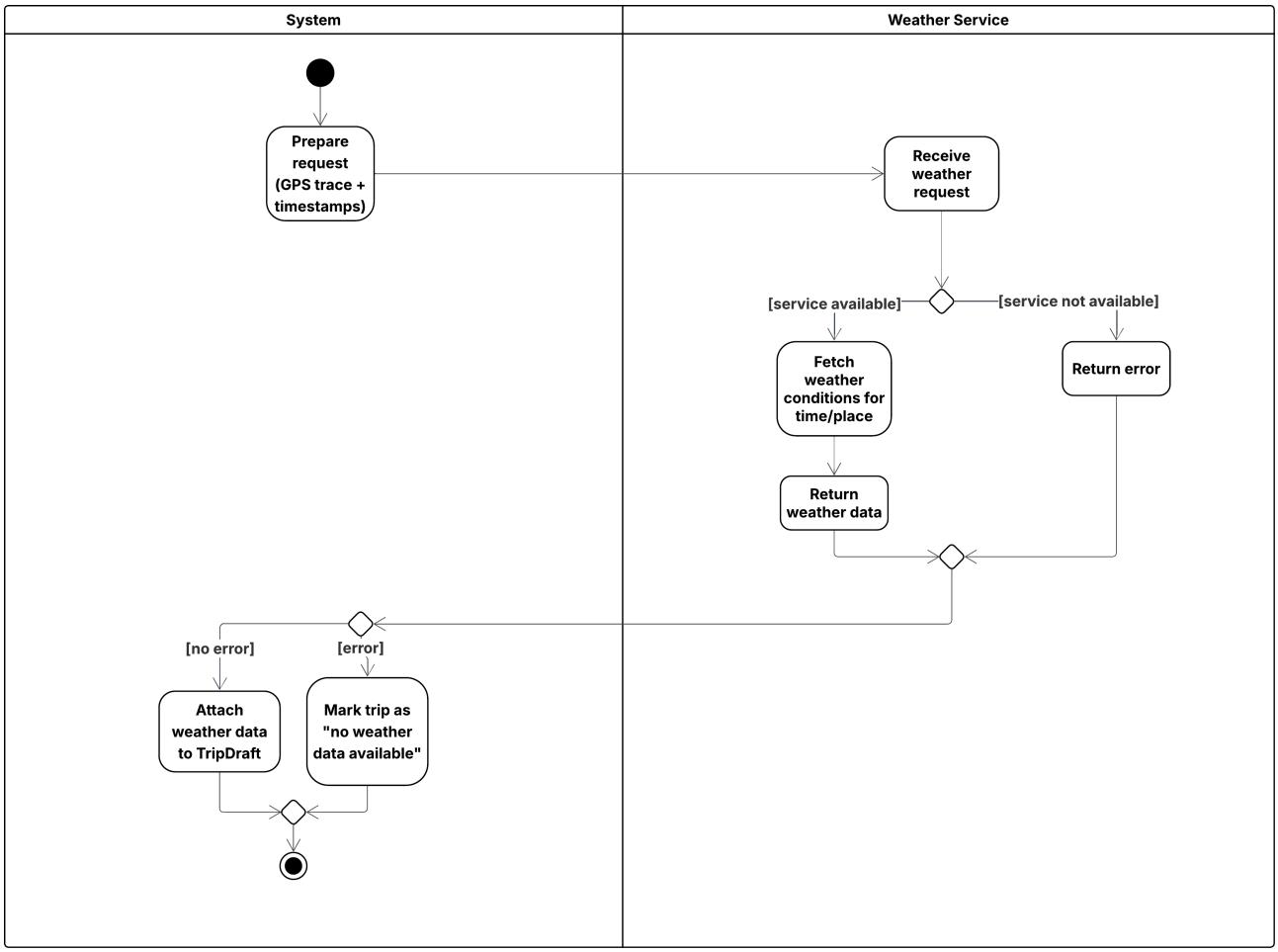


Figure 3.19: Enrich Trip with Weather Information

UC13. Edit Trip

Actor	Registered User
Entry Conditions	A recorded trip (automatic defined) and its associated draft Contributions are available for review (at the end of recording).
Flow of Events	1. The system shows the trip together with the obstacles and Contributions that have been automatically detected. 2. The User inspects the detected obstacles and Contributions and may confirm them, adjust their details, or remove the ones considered incorrect. 3. The User can also add missing Contributions for streets included in the trip by selecting a street and specifying additional information (e.g., a new obstacle or warning). 4. After reviewing the Contributions, the User chooses an action (e.g., store only, store and publish, or discard). 5. If the User chooses to *store and publish*, the system iteratively processes the Contributions associated with the trip: for each Contribution, the system invokes UC. Publish Contribution and then selects the next Contribution until there are no more. 6. Once all Contributions have been processed, the system stores the trip in the User's history together with the final set of Contributions. 7. If the User chooses to *store* without publishing, the system skips the publication step and only stores the trip and its Contributions in the User's history as private data. 8. If the User decides to *discard*, the system deletes the trip data and all associated draft Contributions, terminating the use case.
Exit Conditions	The trip is either stored (with or without published Contributions) in the User's history, or it has been completely discarded.
Exceptions	• An error occurs while storing the trip or its Contributions: the system notifies the User of the failure and suggests retrying the operation; no partial or inconsistent trip data is left saved. • An error occurs during the publication of a Contribution (in the included UC Publish Contribution): the system notifies the User and leaves the affected Contribution in a non-published state while still allowing the rest of the trip to be stored.

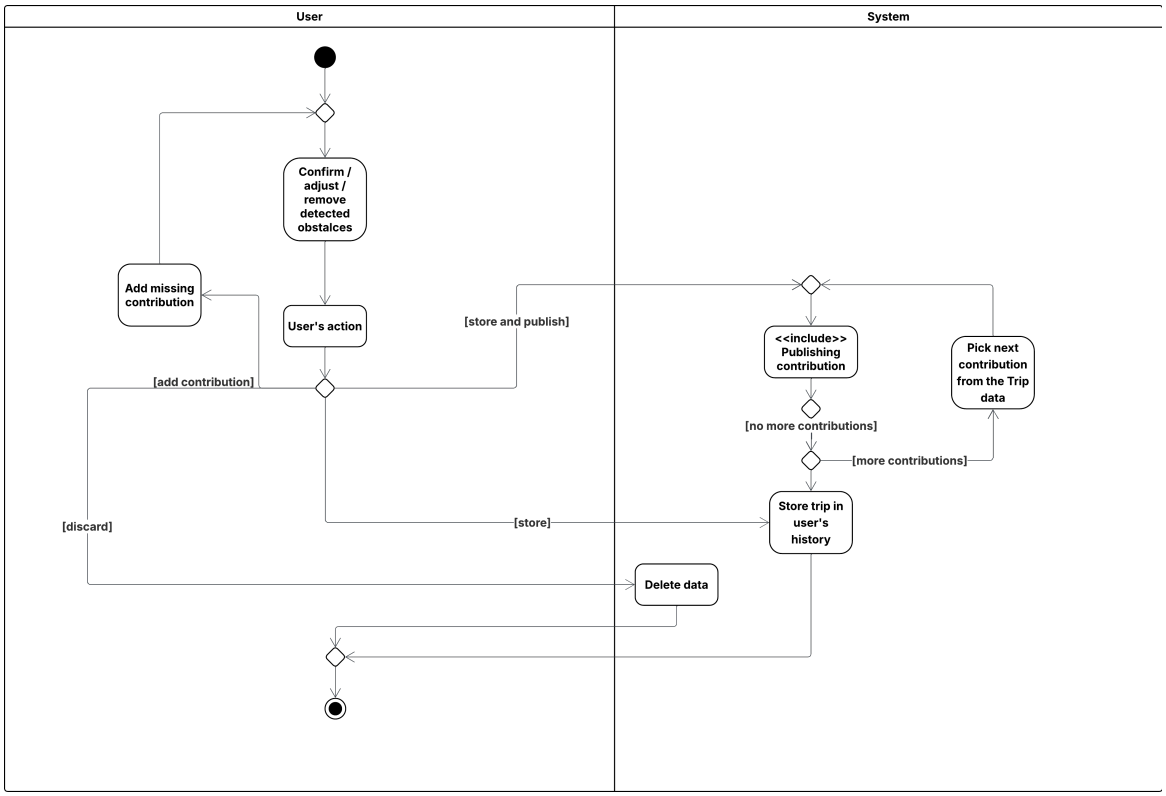


Figure 3.20: Edit Trip

UC14. Follow Path (extends UC. Automatic Insertion)

Actor	Registered User
Entry Conditions	The User has selected a path from the map (see UC. View Map) and chooses to follow it.
Flow of Events	1. The system highlights the selected path on the map and opens a dedicated "Follow path" view. 2. The User starts recording, triggering UC. Automatic Insertion for the current session. 3. While the recording is active, the map keeps the chosen path visually highlighted; the User is free to follow it or deviate from it, and the system does not recompute or enforce the route. 4. All other behaviours (trip recording, obstacle inference, interruption detection, reconstruction of the path, statistics computation and subsequent review/edit) follow exactly the flow described in UC. Automatic Insertion.
Exit Conditions	An automatic trip has been recorded while a selected path was highlighted on the map; the resulting trip is then handled as in UC. Automatic Insertion (review, edit, store, publish).
Exceptions	• Sensors or GPS are not available: the system notifies the User and does not start the follow-path recording.

Remark. Since the behaviour of this use case is almost identical to UC. Automatic Insertion, apart from the highlighted path on the map and the absence of route recalculation, no dedicated activity diagram is provided.

3.2.2. Mapping On Requirements

Row ID	Goal ID	Req ID	DA ID	UC ID
1	G1	R1, R2, R3, R4	DA3, DA4, DA5	UC1, UC2, UC3, UC4, UC5
2	G2	R2, from R5 to R13, R35, R36	DA1, DA2, DA3, DA6, DA7, DA8, DA9, DA12,	from UC10 to UC14

Row ID	Goal ID	Req ID	DA ID	UC ID
3	G3	R7, R8, R10, R15, R35, R36	DA1, DA2, DA6, DA7, DA9, DA10, DA11	UC6, UC13
4	G4	R2, R10, R11, R12, R13	DA3, DA12, DA13	UC10, UC13
5	G5	R2, R5, R6, R7, R8, R9, R10, R11, R35, R36	DA3, DA6, DA7, DA9 DA10, DA11,	UC11, UC12, UC14
6	G6	R2, R14, from R16 to R23	DA3	UC7
7	G7	from R24 to R34	DA1, DA2, DA3, DA11	UC8, UC9
8	G8	R19, R20, R21, R22, R23, R33, R34	DA1, DA2, DA14, DA15	UC7

3.3. Performance Requirements

The system gives real time information to its users, so it is critical that it provides a smooth and responsive user experience on mobile devices under normal operating conditions therefore:

- The system shall provide a **responsive user interface**, ensuring that standard user interactions (e.g., navigation between screens, map panning and zooming, profile access) are completed within **0.5 seconds** under normal network conditions.
- Map rendering and updates, including the visualization of bike paths and aggregated contributions, shall be completed within **2 seconds** after the user changes the visible map area or performs a path search.
- The system shall support **real-time trip recording**, continuously collecting GPS and sensor data without causing noticeable delays, application freezes, or data loss during normal usage.
- Sensor data acquisition (e.g., GPS, accelerometer, gyroscope) shall not exceed acceptable latency thresholds that could compromise the accuracy of recorded trips or obstacle detection.
- The system shall be capable of handling **concurrent access by 15000 users** without degradation of core functionalities, such as map visualization, path search, and trip recording.
- Data synchronization between the mobile application and the backend server shall occur

within a reasonable time frame, ensuring that newly recorded trips and contributions are available to the user within **30 seconds** after successful upload.

- The system shall ensure acceptable performance under varying network conditions, gracefully handling temporary connectivity issues by caching essential data locally and synchronizing it once the connection is restored.

3.4. Design Constraints

3.4.1 Standards Compliance

- The system shall be developed in accordance with the guidelines and best practices for native mobile applications provided by **Android** and **iOS** platforms, including the official Android Developer Guidelines and Apple Human Interface Guidelines.
- The system shall comply with the **ISO/IEC/IEEE 29148:2018** standard for requirements engineering, ensuring consistency and traceability between requirements and design decisions.
- All data exchanged between the mobile application and backend services shall use secure communication protocols compliant with **HTTPS** and **TLS** standards.
- The system shall follow recognized **mobile security best practices**, including secure credential handling, encrypted local storage where required, and protection against common vulnerabilities such as unauthorized data access.
- The system shall adhere to applicable **data protection and privacy regulations**, including the **GDPR**, ensuring transparency in the collection, storage, and processing of personal and location-based data.
- The system shall comply with standard accessibility guidelines for mobile applications, such as the **WCAG 2.1** principles where applicable, to ensure usability for users with different abilities.

3.4.2 Hardware Limitations

- The system shall operate within the hardware constraints typical of mobile devices, including limited CPU resources, memory, storage capacity, and battery life.
- The application shall be optimized to function correctly on devices with heterogeneous hardware configurations, ensuring acceptable performance on both high-end and mid-range smartphones.

- Continuous use of hardware components such as **GPS**, **accelerometer**, and **gyroscope** shall be managed efficiently to minimize battery consumption and prevent excessive resource usage.
- The system shall handle cases in which required hardware components are unavailable, disabled or denied by the user by providing appropriate feedback and gracefully degrading affected functionalities.

3.5. Software System Attributes

3.5.1 Reliability

- The system shall ensure correct and consistent behavior during normal operation, minimizing unexpected crashes or data loss.
- The system shall preserve recorded trip data and user contributions in the presence of temporary failures, such as application restarts or network interruptions.
- The system shall detect and handle runtime errors, providing informative feedback to the user when an operation cannot be completed.

3.5.2 Availability

- The system shall be available for use whenever the mobile device is operational and has network connectivity.
- Core functionalities such as trip recording and data visualization shall remain usable even during temporary backend unavailability, relying on local storage and deferred synchronization.
- The system shall support background execution, where permitted by the operating system, to ensure continuity of trip recording.

3.5.3 Security

- The system shall protect user data by enforcing authentication mechanisms and secure session management.
- Sensitive information, including credentials and personal data, shall be transmitted exclusively through encrypted channels using HTTPS and TLS.
- The system shall restrict access to user-specific data, ensuring that each user can only

view and modify their own information.

- The system shall request explicit user consent before accessing sensitive hardware components, such as GPS and motion sensors.

3.5.4 Maintainability

- The system shall be designed following modular and layered architectural principles to facilitate maintenance and future enhancements.
- The codebase shall be documented and structured to support debugging, testing, and incremental updates.
- The system shall allow updates and bug fixes to be deployed without requiring changes to user data or system reconfiguration.

3.5.5 Portability

- The system shall be portable across the supported mobile platforms, namely Android and iOS.
- Platform-specific components shall be isolated to minimize the effort required to adapt the application to different operating system versions.
- The system shall support future migration to additional mobile platforms with limited changes to the core application logic.

4 | Formal Analysis using Alloy

4.1. Objectives

The formal modeling activity has been carried out using Alloy 6 with the purpose of validating and ensuring the correctness of the core processes of the Best Bike Paths (BBP) system. The primary objectives of this modeling activity are the following:

4.1.1. Consistent System Evolution

We have modeled the rules that define the system's dynamic evolution over time. For each action, we have specified the preconditions that must hold for its execution and the postconditions that describe its effects. Frame conditions have been used to guarantee that elements not involved in an action remain unchanged. In particular, we focused on the following critical actions:

- Saving a trip by a registered user
- Publishing a contribution
- Linking a contribution to a trip
- Confirming a detection
- Merging contributions

By defining these actions carefully, we ensured that the system's evolution remains consistent and predictable under all possible sequences of interactions.

4.1.2. Contribution and Detection Management Logic

We have formalized how contributions and detections are managed in the system, including their creation, association with trips, publication, and eventual merging. The model guarantees that each contribution is linked to exactly one trip and one author, and that detection confirmations correctly generate contributions without introducing inconsistencies.

These parts of the system are the most critical, as they capture the complex interactions between registered users, trips, and the dynamic data collected from the environment. To keep the model tractable and understandable, we intentionally omitted less critical aspects, such as user deletion or detailed weather data processing, focusing on the key functionalities needed to validate system behavior.

Note: The merging logic has been simplified for the sake of clarity; specifically, the automatic

transformation of `BikeSuitability` information was decoupled from the primary `merge` predicate due to its complexity. However, the impact of these suitability updates on the environment was still rigorously verified within the final functional scenario (`PathBreakScenario`).

4.2. The Model

4.2.1. Modular Structure

The Alloy model of BBP has been organized modularly by separating **signatures**, **predicates**, **facts**, and **main scenarios** into different files:

- **Signatures (signatures.als)** define the structural elements of the system, including users, trips, contributions, detections, street segments, paths, and supporting entities such as weather information. Enum types like `Score` capture qualitative attributes of streets and contributions.
- **Predicates (predicates.als)** describe the core actions of the system. Each predicate specifies the preconditions for executing the action, the postconditions describing the resulting state, and the frame conditions that ensure elements not affected by the action remain unchanged.
- **Facts (facts.als)** enforce the consistency of the world. They guarantee, for instance, that each contribution belongs to exactly one trip, authorship is consistent, paths are topologically connected, and scores are correctly propagated. Facts ensure that the system evolves only according to the defined actions, preventing spontaneous or invalid state changes.
- **Main module (main.als)** defines the initial state of the system and models its evolution over time. It also contains test scenarios to validate typical interactions, such as saving a trip, publishing contributions, confirming detections, and merging contributions. The `doNothing` predicate ensures idle transitions preserve consistency, while the `System` predicate allows all valid evolutions according to the defined actions.

This modular organization allows for clear reasoning about each aspect of the system and facilitates automatic analysis using Alloy. The model provides a foundation to verify that BBP behaves as intended under different sequences of user interactions and environmental inputs.

4.2.2. Signatures

This file contains the definitions of the Signatures of the main entities in the Best Bike Paths (BBP) system. BBP supports users in recording and browsing bike paths, either manually or automatically. The main entities modeled are Users (Registered and Unregistered), Trips (Manual and Automatic), Detections, Contributions, Paths, Street Segments, and auxiliary types such as WeatherInfo and Score. Fields have been defined using `one` or `set` cardinalities to avoid unnecessary complexity and improve model performance. Constraints for each Signature are defined to ensure that instances are always meaningful and consistent.

Listing 4.1: Alloy Signatures

```
module signatures

open util/boolean

// --- BASIC ENTITIES ---

sig Coordinate {}

one sig WeatherService {}

sig WeatherInfo {}

// --- USERS ---

var abstract sig User {}

var sig UnregisteredUser extends User {}

var sig RegisteredUser extends User {
    // Dynamic relations tracking a user's activity
    var contributions: set Contribution,
    var trips: set Trip
}

// --- TRIPS ---

var abstract sig Trip {
```

```
    var contributions: set Contribution,
    var madeOf: set StreetSegment
}

var sig ManualTrip extends Trip {}

var sig AutomaticTrip extends Trip {
    // Automatic trips include external weather data and sensor detections
    var weatherData: lone WeatherInfo,
    var detections: set Detection
}

// --- DETECTIONS ---

var sig Detection {
    var detectedInfo: one Information,
    var onStreet: one StreetSegment,
    var confirmed: one Bool,
    var score: one Score
}

// --- INFORMATION ---

var abstract sig Information {
    var position: one Coordinate
}

var sig Warning extends Information {}
var sig Event extends Information {}

var abstract sig BikeSuitability extends Information {
    var isSuitable: one Bool
}

var sig SpeedLimUpdate extends BikeSuitability {}
var sig BikeLaneUpdate extends BikeSuitability {}
var sig HeavyTraffic extends BikeSuitability {}

// --- CONTRIBUTIONS ---
```

```
var sig Contribution {
    var author: lone RegisteredUser,
    var segment: one StreetSegment,
    var info: set Information,
    var score: one Score
}

// Subset of contributions that have been verified and integrated into the map
var sig PublishedContribution in Contribution {}

// --- PATH GENERATION ---

one sig PathGenerate {
    var activePaths: set Path
}

sig Path {
    // Every path must consist of at least one segment
    segments: some StreetSegment,
    score: one Score
}

// --- STREET SEGMENTS ---

sig StreetSegment {
    start: one Coordinate,
    end: one Coordinate,
    // Attributes that change based on user feedback and processing
    var score: one Score,
    var isCyclable: one Bool
}

// --- ENUMERATIONS ---

enum Score { Optimal, Medium, Sufficient, RequiresMaintenance, None }
```

4.2.3. Predicates

This file contains the modeling of the main operations that users can perform in the Best Bike Paths (BBP) system, expressed as Alloy predicates. Each predicate defines preconditions that must hold for the operation to execute, and postconditions that specify the changes in the system state visible in the next instant. Frame conditions are also included to ensure that unrelated parts of the system remain unchanged.

Predicates cover the main functionalities of BBP for managing trips, contributions, and detections, including:

- **saveTrip**: Records a new trip for a registered user.
- **publishContribution**: Publishes a contribution so that it becomes visible to all users.
- **saveContribution**: Links a contribution to a trip and its author.
- **confirmDetection**: Confirms an automatically detected information and converts it into a contribution.
- **merge**: Updates the score of a street segment based on a contribution.

Listing 4.2: Alloy Predicates for BBP

```
module predicates

open signatures

// --- SAVE TRIP ---

pred saveTrip[u: RegisteredUser, t: Trip] {
    // Preconditions
    u in RegisteredUser
    all u1: RegisteredUser | t not in u1.trips

    // Postconditions
    u.trips' = u.trips + t
    u.contributions' = u.contributions + t.contributions

    all u2: RegisteredUser - u | u2.trips' = u2.trips

    // Frame conditions
```

```
    (Trip <: madeOf)' = (Trip <: madeOf)
    (Trip <: contributions)' = (Trip <: contributions)
    (Trip <: detections)' = (Trip <: detections)

    User' = User
    Contribution' = Contribution
    PathGenerate.activePaths' = PathGenerate.activePaths
    (StreetSegment <: score)' = (StreetSegment <: score)
}

// --- PUBLISH CONTRIBUTION ---

pred publishContribution[u: RegisteredUser, c: Contribution] {
    // Preconditions
    u in RegisteredUser
    c not in PublishedContribution
    c.author = u

    // Postconditions
    PublishedContribution' = PublishedContribution + c

    // User and Author updates
    (RegisteredUser <: contributions)' = (RegisteredUser <: contributions) + (u ->
        c)
    (c.author)' = u
    some c.(Contribution <: score)'

    // Frame conditions
    User' = User
    Trip' = Trip
    Information' = Information

    trips' = trips
    madeOf' = madeOf
    activePaths' = activePaths
    weatherData' = weatherData
    (StreetSegment <: score)' = (StreetSegment <: score)
}
```

```
// --- SAVE CONTRIBUTION (LINKING) ---

pred saveContribution[u: RegisteredUser, t: Trip, c: Contribution] {
    // Preconditions: the contribution must not be already linked
    all u1: RegisteredUser | c not in u1.contributions
    all t1: Trip | c not in t1.contributions

    // Postconditions: establish the link between contribution, author, and trip
    c.author' = u
    t.contributions' = t.contributions + c

    // Frame conditions
    User' = User
    Contribution' = Contribution
    PathGenerate.activePaths' = PathGenerate.activePaths
    trips' = trips
}

// --- CONFIRM DETECTION ---

pred confirmDetection[u: RegisteredUser, t: AutomaticTrip, d: Detection, c:
    Contribution] {
    // Preconditions: detection must be unconfirmed and belong to the user's trip
    all u1: RegisteredUser | c not in u1.contributions
    all t1: Trip | c not in t1.contributions
    d in t.detections
    d.confirmed = False
    t in u.trips

    // Postconditions: confirm the detection and save it as a contribution
    d.confirmed' = True

    c.segment = d.onStreet
    c.info = d.detectedInfo
    c.score = d.score

    saveContribution[u, t, c]

    // Frame conditions
```

```
    User' = User
}

// --- MERGE CONTRIBUTION ---

pred merge[c: Contribution] {
    // Preconditions: only published contributions are merged into the global map
    c in PublishedContribution

    // Postconditions: update the segment's score based on the contribution data
    (c.segment.score)' = c.score

    // Frame conditions
    User' = User
    Trip' = Trip
    Contribution' = Contribution
    madeOf' = madeOf
    activePaths' = activePaths
    PublishedContribution' = PublishedContribution
}
```

4.2.4. Facts

This file contains facts for enforcing the consistency of the Best Bike Paths (BBP) system. Facts ensure that the evolution of the system occurs only through the execution of the actions defined in the predicates, and not spontaneously. They impose constraints on the relationships among trips, contributions, detections, street segments, paths, and users, maintaining the internal consistency of the system over time.

Listing 4.3: Alloy Facts for BBP

```
module facts

open signatures
open predicates

// --- STRUCTURAL CONSISTENCY ---
```

```
fact StructuralConsistency {
    // Each Trip belongs to at most one RegisteredUser
    all t: Trip | lone u: RegisteredUser | t in u.trips

    // Each Contribution belongs to at most one Trip
    all c: Contribution | lone t: Trip | c in t.contributions
}

// --- DETECTIONS & INFORMATION CONSISTENCY ---

fact detectionConsistency {
    // Detections associated with a trip persist in the next state
    all t: Trip, d: Detection |
        d in t.detections implies d in (t.detections)'
}

fact detectedInfoConsistency {
    // Information linked to a detection persists in the next state
    all i: Information, d: Detection |
        i in d.detectedInfo implies i in (d.detectedInfo)'
}

fact streetConsistency {
    // The street segment associated with a detection remains the same over time
    always all s: StreetSegment, d: Detection |
        s in d.onStreet implies s in (d.onStreet)'
}

fact detectionTripConsistency {
    // A single detection cannot belong to two different automatic trips
    always all disj t1, t2: AutomaticTrip | no t1.detections & t2.detections
}

// --- AUTHOR CONSISTENCY ---

fact authorConsistency {
    // If a contribution is in a user's set, that user must be the author
    always all c: Contribution, u: User |
        c in u.contributions implies c.author = u
```

```
}

fact AuthorConsistency {
    // The author of a contribution must be the owner of the trip it belongs to
    always all u: RegisteredUser, t: u.trips, c: t.contributions |
        c.author = u
}

// --- TOPOLOGICAL CONSISTENCY ---

fact TopologicalConsistency {
    // A contribution must refer to a segment that is part of the original trip
    always all t: Trip, c: t.contributions |
        c.segment in t.madeOf
}

// --- PATH CONSISTENCY ---

fact PathConnectivityAndScore {
    // Geometric connectivity : segments in a path must be physically connected
    all p: PathGenerate.activePaths | {
        #p.segments > 1 implies all s: p.segments |
            some s2: p.segments - s |
                (s.end = s2.start) or
                (s.start = s2.end) or
                (s.end = s2.end) or
                (s.start = s2.start)
    }

    // Path score is determined by the minimum score among its segments
    all p: PathGenerate.activePaths | {
        some p.segments implies p.score = min[p.segments.score]
    }
}

fact PathMustBeValid {
    // Active paths must only consist of cyclable segments
    always all p: PathGenerate.activePaths |
        all s: p.segments | s.isCyclable = True
```

```
}

fact PathValidityDefinition {
    // The set of activePaths contains exactly those paths where all segments are cyclable
    always PathGenerate.activePaths = { p: Path | all s: p.segments | s.isCyclable =
        True }
}

// --- USER & HISTORY CONSISTENCY ---

fact ConsistentUsersSet {
    // The set of users remains constant over time
    always (
        User = User'
    )
}

fact ConsistentHistory {
    // A user's trip history is preserved unless a new trip is saved
    always (
        all u: RegisteredUser | (
            (u.trips = u.trips')
            or (some t: Trip | saveTrip[u, t])
        )
    )
}

fact ConsistentContributions {
    // A user's contributions are preserved unless a new one is published
    always (
        all u: RegisteredUser | (
            (u.contributions = u.contributions')
            or (some c: Contribution | publishContribution[u, c])
        )
    )
}

// --- CONTRIBUTION CONSISTENCY ---
```

```
fact ConsistentContributionScore {
    // The score assigned to a contribution remains constant
    always all c: Contribution |
        c.score = (c.score)'
}

fact ConsistentContributionStreet {
    // The street segment linked to a contribution remains constant
    always all c: Contribution |
        c.segment = (c.segment)'
}

fact suitabilityLimitation {
    // A contribution cannot contain multiple distinct bike suitability updates
    all c: Contribution, i1, i2: BikeSuitability |
        (i1 in c.info and i2 != i1) implies i2 not in c.info
}

fact publishedConsistency {
    // Once a contribution is published, it remains published in subsequent states
    always all c: PublishedContribution |
        c in PublishedContribution'
}
```

4.2.5. Main Module

This file defines the overall configuration and evolution of the Best Bike Paths (BBP) system. It sets the initial state, specifies static constraints on the topology of the world, and models the system evolution over time. The module also defines test scenarios to illustrate how users interact with the system, such as saving trips, publishing contributions, and confirming detections.

Each predicate in this module either sets up an initial state, models a do-nothing transition to maintain consistency, or represents possible system evolutions by invoking the previously defined actions (predicates). Test scenarios are included to validate the expected behavior of BBP in specific use cases.

Listing 4.4: Alloy Main Module for BBP

```
module main

open signatures
open facts
open predicates

// --- 1. CONFIGURATION & INITIAL STATE ---

// World topology constraints (minimum requirements for instance generation)
fact WorldTopology {
    #StreetSegment >= 2
    all s: StreetSegment | some s.start and some s.end
}

// Definition of the initial state at time T=0
pred initialize {
    // No users or data present at startup
    no RegisteredUser
    no UnregisteredUser

    all u: RegisteredUser | no u.trips
    all u: RegisteredUser | no u.contributions
    all t: Trip | no t.contributions

    no PathGenerate.activePaths

    // All segments start with an optimal score
    all s: StreetSegment | s.score = Optimal
}

// --- 2. TRANSITIONS & SYSTEM EVOLUTION ---

// "Idle" transition: the system state remains unchanged
pred doNothing {
    RegisteredUser' = RegisteredUser
    UnregisteredUser' = UnregisteredUser

    trips' = trips
```

```
    madeOf' = madeOf
    activePaths' = activePaths

    (Trip <: contributions)' = (Trip <: contributions)
    (RegisteredUser <: contributions)' = (RegisteredUser <: contributions)

    (StreetSegment <: score)' = (StreetSegment <: score)
    (Contribution <: score)' = (Contribution <: score)

    weatherData' = weatherData
}

// Dynamic evolution of the system over time
pred System {
    initialize
    and
    always (
        doNothing
        or
        (some u: RegisteredUser, t: Trip | saveTrip[u, t])
        or
        (some u: RegisteredUser, c: Contribution | publishContribution[u, c])
        or
        (some c: Contribution | merge[c])
    )
}

// --- 3. TEST SCENARIO: SAVE TRIP + CONTRIBUTION ---

// Verifies the workflow: Save Trip -> Publication -> Data Merge
pred TestScenario {
    some u: RegisteredUser,
         t: Trip,
         c: Contribution,
         s: StreetSegment | {

        // Static data setup for the test
        s.score = Optimal
        c.score = RequiresMaintenance
```

```
        // Initial test state
        no u.trips
        no PublishedContribution & c

        // Transition 1: User saves a manual trip
        saveTrip[u, t]
        s in t.madeOf'
        t in ManualTrip

        // Transition 2: Publication of the contribution related to segment s
        after publishContribution[u, c]
        c.segment = s
        c in t.contributions

        // Transition 3: Integration of feedback into the system (Merge)
        after after merge[c]
    }
}

// Command to verify the consistency of the test scenario
run TestScenario for 4 but 4 Int, 4 seq

// --- 4. TEST SCENARIO: DETECTION CONFIRMATION ---

// Verifies the transformation of an automatic Detection into a user contribution
pred TestDetectionConfirmation {
    some u: RegisteredUser,
         t: AutomaticTrip,
         d: Detection,
         s: StreetSegment,
         c: Contribution | {

        // Static links between entities
        d.onStreet = s
        d in t.detections
        t in u.trips

        // Contribution must be initially orphan
```

```
        no c.author
        all existingT: Trip | c not in existingT.contributions

        // Initial state: detection not yet confirmed
        d.confirmed = False
        no t.contributions

        // Transition: User confirms the automatic detection
        confirmDetection[u, t, d, c]

        // Verify post-condition in the next state
        after {
            d.confirmed = True
            one newC: Contribution | {
                newC in t.contributions
                newC.author = u
                newC.segment = s
                newC.score = d.score
            }
        }
    }
}

// run TestDetectionConfirmation for 2 but 2 Contribution, 2 seq

// --- 5. TEST SCENARIO: PATH INVALIDATION (SAFETY) ---

// Demonstrates that a segment failure only invalidates paths containing it
pred PathBreakScenario {
    some pAffected, pSafe: Path, s: StreetSegment | {

        // --- STRUCTURAL SETUP ---
        pAffected != pSafe          // Paths must be distinct

        s in pAffected.segments     // s is critical only for pAffected
        s not in pSafe.segments     // pSafe does not depend on s

        // Intersection constraint: paths share another segment (sShared)
        some sShared: StreetSegment | {
```

```
        sShared in pAffected.segments
        sShared in pSafe.segments
    }

    #pAffected.segments > 1
    #pSafe.segments > 1

    // --- TIME 0: Healthy operational state ---
    pAffected in PathGenerate.activePaths
    pSafe in PathGenerate.activePaths

    s.isCyclable = True
    all seg: (pAffected.segments + pSafe.segments) | seg.isCyclable = True

    // --- TRANSITION: Segment failure ---
    s.isCyclable' = False

    // Frame condition: the only change is the state of s
    all s2: StreetSegment - s | s2.isCyclable' = s2.isCyclable

    // --- TIME 1: Verify path independence ---
    after {
        // pAffected is removed because s is no longer cyclable
        pAffected not in PathGenerate.activePaths

        // pSafe remains active as it does not contain s
        pSafe in PathGenerate.activePaths

        s.isCyclable = False
    }
  }
}

// run PathBreakScenario for 4 but 2 seq, 8 StreetSegment, 8 Coordinate
```

4.3. Validation through the Model

This section presents the formal validation of the system's dynamic behavior using the Alloy Analyzer. The following instances illustrate the "Report a Road Issue" scenario, obtained by executing the `TestScenario` predicate defined in the `main.als` module.

4.3.1. Scenario: User Reporting a Road Issue

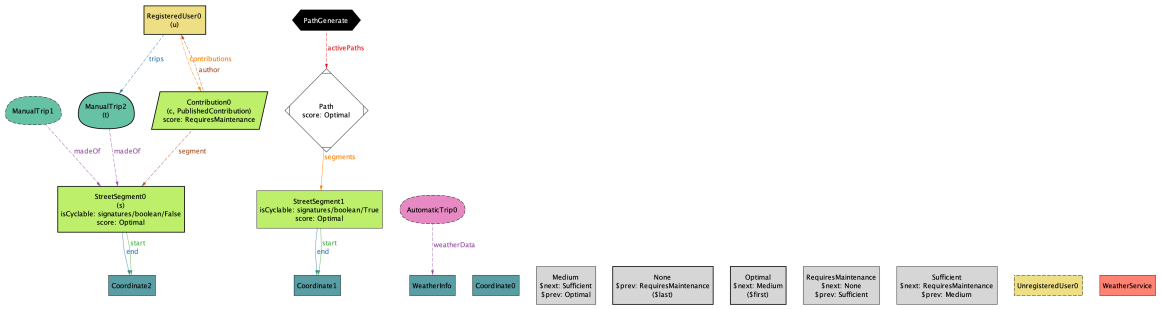


Figure 4.1: Dynamic World, Time 2: Contribution Publication

Figure 4.1 depicts the system state immediately after a Registered User saves a trip and publishes a contribution. The `Contribution0` object records a `RequiresMaintenance` score; however, the associated `StreetSegment0` still retains its initial `Optimal` status, correctly representing the pending state before data processing occurs.

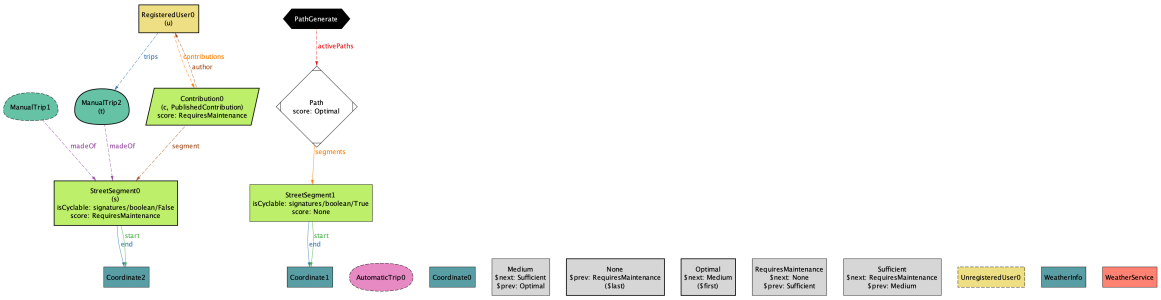


Figure 4.2: Dynamic World, Time 3: Data Merge and Update

Figure 4.2 shows the state after the execution of the `merge` predicate. The system aggregates the feedback, forcing an update on the map topology: `StreetSegment0` adopts the `RequiresMaintenance` score and its `isCyclable` attribute switches to `False`, confirming that the user's report successfully impacted the global knowledge.

4.3.2. Scenario: Automatic Detection Confirmation

This scenario validates the interaction where a user confirms a sensor detection during an automatic trip, transforming a raw system alert into a verified user contribution. The following instances correspond to the `TestDetectionConfirmation` predicate.

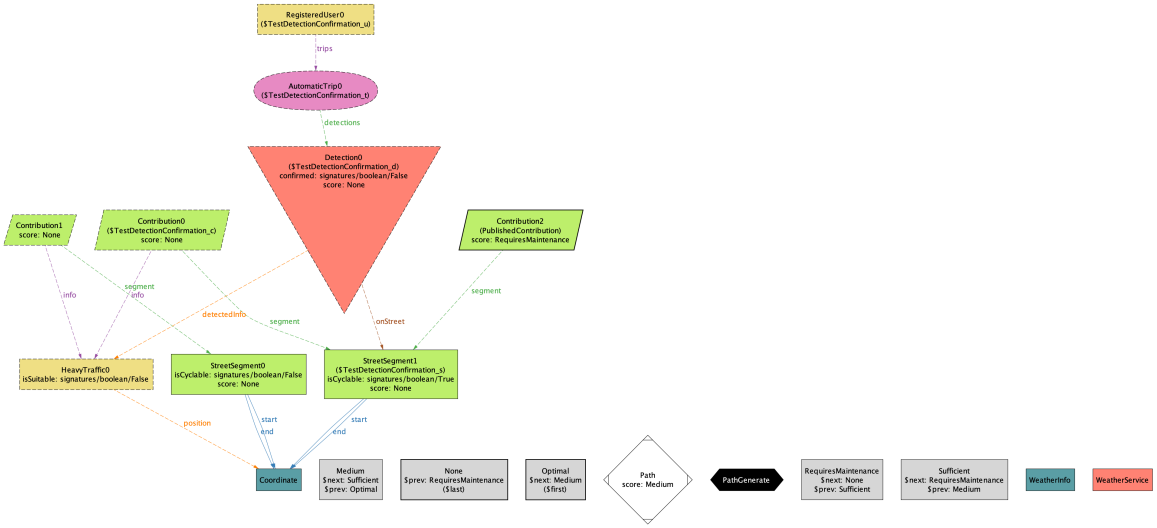


Figure 4.3: Dynamic World, Step 0: Unconfirmed Detection

Figure 4.3 illustrates the initial state of an `AutomaticTrip`. A `Detection` object (red triangle) is linked to the trip and a street segment, but its status is currently unconfirmed (`confirmed: False`). At this stage, the detection exists as raw data, and no formal `Contribution` is yet associated with the user or the trip.

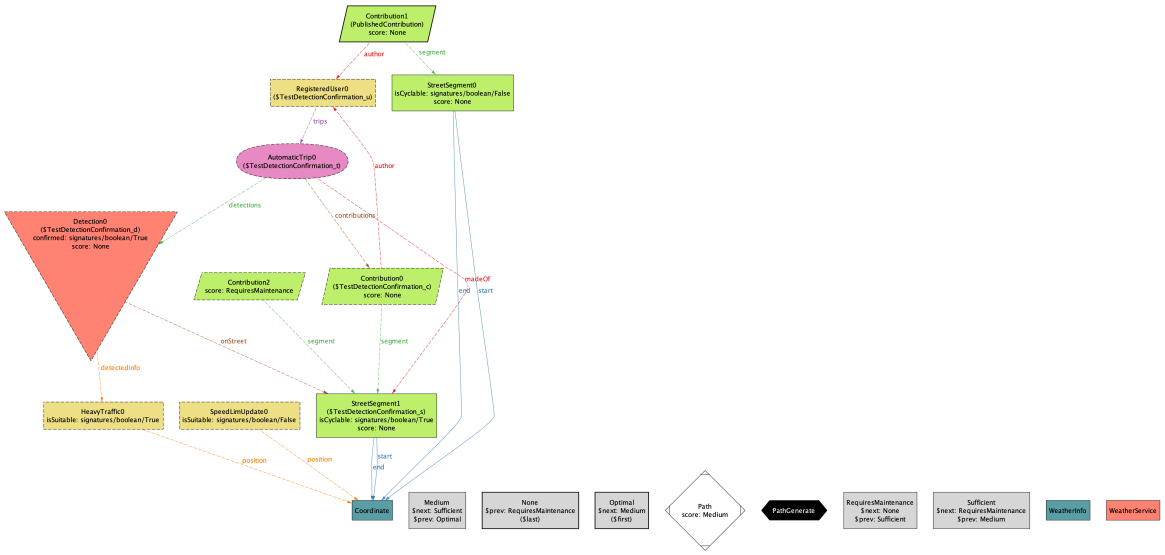


Figure 4.4: Dynamic World, Step 1: Confirmation and Contribution Generation

Figure 4.4 depicts the system state after the `confirmDetection` predicate is executed. The `Detection` status updates to `True`, and the system formally generates a new `Contribution`. This new object is now structurally linked to the `RegisteredUser` and the `AutomaticTrip`, inheriting the score and segment location from the original detection.

4.3.3. Scenario: Path Invalidation and Independence

This scenario demonstrates the system's topological consistency. It verifies that if a specific street segment becomes non-cyclable, only the paths utilizing that specific segment are invalidated, while intersecting or nearby paths remain active. The following instances correspond to the `PathBreakScenario` predicate.

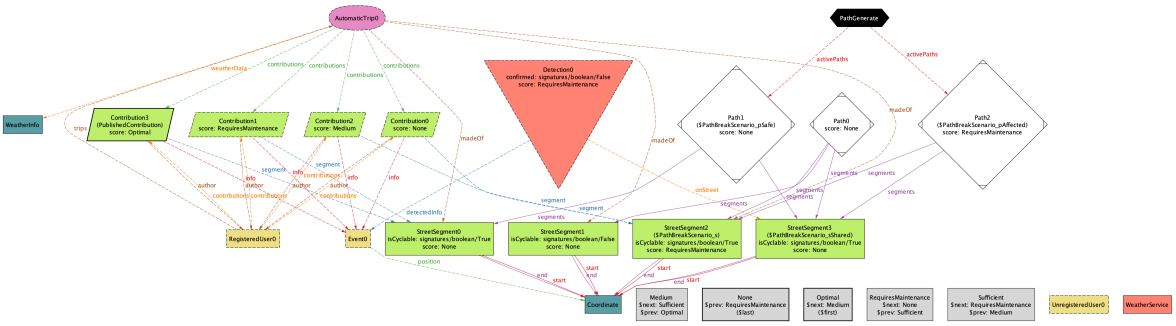


Figure 4.5: Dynamic World, Step 0: Initial Connectivity

Figure 4.5 shows the initial system state with two distinct paths, `Path1` and `Path2` . Both paths are currently valid and included in `PathGenerate.activePaths` because all their constituent segments are cyclable. Notably, the paths share a common geometry (a shared segment), but `Path1` relies on a specific critical segment `s` that `Path2` does not use.

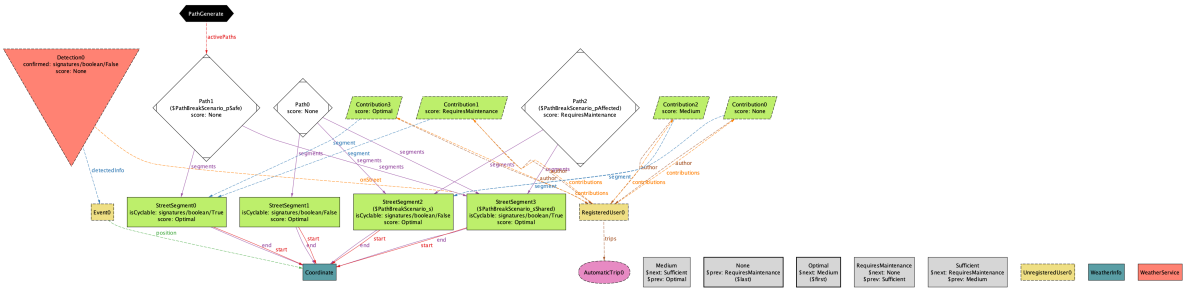


Figure 4.6: Dynamic World, Step 1: Localized Failure

Figure 4.6 depicts the state after the critical segment `s` is marked as non-cyclable (`isCyclable: False`) . The system enforces the validity constraints: `Path1` is immediately removed from

`activePaths`, while `Path2` remains active and unaffected. This proves that issues on a specific road segment are handled locally and do not cascade to invalidate unrelated or merely intersecting valid routes.

5 Effort Spent

Group Member	Effort Spent in each Section	
David Ravelli	Introduction	4*h*
	Overall Description	4*h*
	Specific Requirements	3*h*
	Formal Analysis	14*h*
	Reasoning	8*h*
Francesco Rausa	Introduction	4*h*
	Overall Description	6*h*
	Specific Requirements	13*h*
	Formal Analysis	2*h*
	Reasoning	8*h*
Pietro Poggi	Introduction	4*h*
	Overall Description	6*h*
	Specific Requirements	11*h*
	Formal Analysis	4*h*
	Reasoning	8*h*

Table 5.1: Effort spent by each member of the group.

6 References

6.1. Used Tools

- **Overleaf**: LaTeX collaborative writing.
- **Lucid Chart**: diagrams.
- **Alloy**: formal analysis for the system.
- **ChatGPT**: Used to assist in translating author-defined concepts from Italian to English and in refining the textual presentation of the document. Additional support was provided for LaTeX and Alloy syntax clarification. The content and technical decisions remain entirely the responsibility of the authors. Some notable prompts used were:
  - How do i define the fact that the score of SreetSegments only changes by doing merge[c]? It answered by saying that instead of doing this fact: always (all s: StreetSegment | s.score' = s.score) I should write the frame condition in all predicates but the merge which gave the intended result.
  - Contribution goes from the subset published to a normal contribution in the following state, why? It answered that I forgot to state that the set of published contributions doesn't change outside of the predicate PublishContribution[u,c].
  - "Produce an English definition paragraph for `StreetSegment`. A `StreetSegment` is the atomic entity of the system, it is a road segment between two intersections without internal junctions". The model answered with: " The fundamental unit of the system, representing a stretch of road between two intersections or junctions. Each StreetSegment contains no intersections within it and constitutes the most atomic level of the road network."1.4

[1]

[1]The alloy prompts and answers were paraphrased and translated from Italian for the sake of simplicity.

List of Figures

2.1 Domain-Level Class Diagram. 8
2.2 Domain-Level Class Diagram. 9
2.3 User Registration and Account Management. 11
2.4 Automatic Trip. 12
2.5 Manual Trip. 12
2.6 StreetSegment. 13

3.1 Log in Interface 23
3.2 Map Interface 24
3.3 Profile interface 25
3.4 Automatic trip interface 26
3.5 Detection confirmation interface 27
3.6 Manual trip interface 28
3.7 Use Case Diagram 31
3.8 Sign Up 33
3.9 Sign In 35
3.10 Delete Account 37
3.11 Logout 39
3.12 Edit Profile 41
3.13 View Trip History 43
3.14 Publish Contribution 45
3.15 View Map 48
3.16 Search Path 50
3.17 Manual Insertion 53
3.18 Automatic Insertion 56
3.19 Enrich Trip with Weather Information 58
3.20 Edit Trip 60

4.1 Dynamic World, Time 2: Contribution Publication 84
4.2 Dynamic World, Time 3: Data Merge and Update 84
4.3 Dynamic World, Step 0: Unconfirmed Detection 85
4.4 Dynamic World, Step 1: Confirmation and Contribution Generation 85
4.5 Dynamic World, Step 0: Initial Connectivity 86
4.6 Dynamic World, Step 1: Localized Failure 86